School of Informatics & Cybersecurity

BSc (Hons) Cybersecurity & Digital Forensics

Evaluating the impact of security posture in web environments

Authors: Seán Harmon, Kyle Mcelroy, Chinedu Nkem

Student Number: B00164655,B00164922,B00162316

Module: DFCSH 3018 : Group Project

Lecturer: Mark Lane

Date: Monday,06 May 2026

Word Count: 19,862

Table of Contents

1.0 Introduction 6
2.0 Background 7
2.1 Web Security 7
2.2 Common web attacks 7
2.3 Authentication 10
2.4 User data privacy rights 10
3.0 Literary Review 12
3.1 Broken access control 12
3.2 Authentication and two-factor authentication 13
3.3 Injection and cross-site scripting 14
3.4 Secure coding and testing methodology 15
3.5 Cryptographic failures and client-side data exposure 16
3.6 Insecure design and account enumeration 17
4.0 Methodology 17
4.1 Research Design 17
4.2 Framework Selection: OWASP Top 10:2021 18
4.3 Application Architecture 19
4.3.1 Security controls in the secure application 19
4.4 Testing Environment 20
4.5 Test Selection and Coverage 20
4.5 Shared Testing Protocol 22
4.6 Data Collection and Scoring 23
4.7 Analytical Approach 23
4.8 Threats to Validity 24
4.9 Ethical Considerations 24
5.0 Implementation 25
5.1 Overview 25
5.2 Common Architecture 26
5.2.1 Database schema 26
5.2.2 API structure 27
5.3 Demonstration 28
5.3.1 Setting Up the Environment 28
5.3.2 Installing dependencies 28
5.3.3 Database configuration 28
5.3.4 Starting the server 28
5.4 Demonstration of the Application 29
5.4.1 Login page 29
5.4.2 Two-factor authentication 30
5.4.3 Employee dashboard 31
5.4.4 Administrator panel 32
5.5 Secure Application 33
5.5.1 Password Hashing 33
5.5.2 Parameterised Queries 34

5.5.3 Generic Error Messages 34
5.5.4 Account Lockout 35
5.5.5 Two-Factor Authentication 36
5.5.6 IDOR Protection 36
5.5.7 Admin Access Control 37
5.5.8 HTTP Security Headers 37
5.5.9 Session Cookie Security 38
5.5.10 Resulting state 38
5.6 Naive Application 38
5.6.1 Derivation process 38
5.6.2 What was removed 39
5.6.3 What was deliberately kept 40
5.6.4 Resulting state 41
5.7 Vulnerable Application 41
5.8 Implementation challenges 44
5.8.1 Testing protocol finalised late 44
5.8.2 Framework defaults blurring the security boundary 44
5.8.3 Development errors and use of AI assistance 45
6.0 Analysis 45
6.1 Overview 45
6.2 Results summary 46
6.3 Analysis by OWASP Category 48
6.3.1 A01 — Broken Access Control 48
6.3.1.1 Secure Site 48
6.3.1.2 Naive Site 51
6.3.1.3 Vulnerable Site 53
6.3.2 A02 — Cryptographic Failures 55
6.3.2.1 Secure Site 55
6.3.2.2 Naive Site 56
6.3.2.3 Vulnerable Site 57
6.3.3 A03 — Injection 58
6.3.3.1 Secure Site 58
6.3.3.2 Naive Site 61
6.3.3.3 Vulnerable Site 63
6.3.4 A04 — Insecure Design 65
6.3.4.1 Secure Site 66
6.3.4.2 Naive Site 66
6.3.4.3 Vulnerable Site 67
6.3.5 A05 — Security Misconfiguration 68
6.3.5.1 Secure Site 68
6.3.5.2 Naive Site 70
6.3.5.3 Vulnerable Site 72
6.3.6 A07 — Identification and Authentication Failures 75
6.3.6.1 Secure Site 75
6.3.6.2 Naive Site 78
6.3.6.3 Vulnerable Site 81
6.4 Cross-Cutting Themes 83

7.0 Conclusion 91
7.1 Future Work 92

Abstract

This study is about how a web application's security posture affects exploitability against the OWASP Top 10:2021. Three Flask based payroll web applications were built with identical functionality, the only difference being their security implementation. The secure version included two-factor authentication, parameterised queries, role-based access control, and a custom WAF. The naive version was built without security in mind. The vulnerable version had safe code replaced with unsafe code on purpose. Each application was tested against ten scenarios from the OWASP Web Security Testing Guide v4.2, with failures scored using CVSS v3.1.
The secure version passed every test. The vulnerable version failed every test. The naive version sat between them, passing where framework defaults caught the attack and failing where they did not. Framework defaults covered some of the categories tested but not most, and the categories they missed produced the highest CVSS scores in the study, including broken access control, weak authentication, and missing security headers

Research Question : To what extent does a web application’s security posture affect the exploitability of OWASP Top 10 vulnerabilities?

Problem statement

Despite the need for more complex security systems in the expanding use of web applications that handle sensitive user data , many systems still use a password based authentication system. While this prioritises user convenience to cater to ease of use this can significantly weaken the security posture of a system, leaving it open to critical security vulnerabilities such as the OWASP top 10 vulnerabilities, we will be conducting our evaluation using this list to ensure testing is accurate and fair.

When applied to the system we will be focusing on (Payslip) and financial systems, user data security is of the utmost importance as a breach or leak in that data can have long lasting effects on users should important data be compromised (such as financial data).
For these reasons reliance on a single factor authentication method is no longer sufficient.

Due to this there is a need for stronger authentication methods that go beyond traditional single authentication systems or password based authentication.
This project explores the impact of implementing Multi Factor authentication (2FA) to a security system and comparing this system to a less secure single factor authentication method (password based login) , from this comparisons will be made in attack success rate , time to detection and many more metrics that encompass the security of both systems.

We will be providing a secure website with 2FA and comparing it to an insecure site , measuring attack surface, success rate , methods and more metrics in order to give an all encompassing report to compare the site's security.

1.0 Introduction

Current systems using a password only approach creates security risks, this can leave user data and the system itself to be vulnerable to exploitation and exposure. This project's aim is to provide a clear and comprehensive comparison between 3 sites with varying levels of security, measuring security and attack surface when compared between a secure-by-design system, insecure system and multi factor authentication systems.

Our solution to this is to provide a payslip system using all of the secure programming practices on our baseline site as well as having 2FA implemented on our secure site.
We will conduct these tests using the OWASP top ten vulnerabilities.

The handling of sensitive data is imperative when handling such information,

As our system will be a payslip portal this means we will be handling very sensitive data such as employee personal information (such as D.O.B , address, name etc.) as well as payment information (salary and banking information).
When working with data of this kind it is the role of the developers to prevent this data from being breached by anyone with malicious intent or insufficient credentials.

2.0 Background

2.1 Web Security

Web applications are used for almost all modern organisations to manage information for any stakeholders in the organisations , from employees to investors to customers.
With the reliance on these systems increasing so is the risk of cyberattacks against them,
With this comes the focus on organisations to have strong web security, to protect their users, data and systems from attacks.

Our project focuses on improving the security of a payslip management system by incorporating Two-Factor Authentication (2FA), this provides an additional layer of security and authentication to a traditional password based login system, increasing security and reducing risk to data breaches and exploitation.

Web security is the practice of protecting these systems from unauthorised access, this can be through active methods such as penetration testing (attempting to break a system to ensure it is secure) to ensure system security or more passive approaches such as using secure programming practices in the creation of the system to remediate threats proactively (for example url crawling and gaining access to a page a user shouldn't have access to)

2.2 Common web attacks

Brute force attacks: This is a hacking method that uses trial and error in order to crack passwords, login credentials and encryption keys. The hacker tries many usernames or passwords using the known format for these fields (such as a password with 6 chars and 1 special char, in brute forcing it would use every combination of 6 chars and 1 special char) to forcefully gain access to an account.

Dictionary attacks: A dictionary attack is a basic form of brute forcing in which a malicious user checks a list of compromised passwords against a specific user/username, this type of attack is quite time consuming when compared to other methods and yields a low success rate against other methods.

Credential stuffing: This attack is contingent on poor password practices. The attackers collect username and password combinations which have been previously stolen , which they then test on other websites to try to gain access.
This approach is successful if a user maintains the same passwords and login information for different sites. These can be especially dangerous as they can escalate into identity driven attack patterns which can lead to a user being logged out of multiple platforms leaving the malicious user to assume the identity of the victim.
This attack method can be mediated by having secure password practices and maintaining different passwords for each platform.

Phishing: This type of attack uses social engineering and fraudulent emails , messages or phone calls to trick people into sharing sensitive data that the attacker should not have access to, therefore exposing themselves or the system to cybercrime, there are many different types of phishing attacks such as spear phishing (where one specific user is targeted for these attacks) and angler phishing where the attacker poses as a customer service member in order to gain the trust of the individual.

These approaches are facilitated by the use of social engineering as opposed to system structure, whereas other security approaches involve changing the code base social engineering preys on the gullibility of a user with access to the system (for example a seemingly important email with a malicious link), which can then be exploited by a malicious user .

These attacks can be prevented using 2FA so that if an account password is breached the trusted device or email is required to complete authentication.

SQL injection: This is a type of attack that allows malicious users to pull data from the system database in order to extrapolate sensitive data using methods such as fetching a user profile or updating a product listing,
however with SQLi an attacker can inject malicious SQL queries into input fields like search bars or login pages, this happens because the system is too trusting and is treating the injected code as text allowing data to be pulled.
This is why a zero trust system is more secure.

Error-based SQLi : This relies on the database returning error messages, this is because the error messages can be revealing when it comes to database information such as table names or the structure of queries, allowing the malicious users to input their own fields into the format to exploit the database.

Union-based SQLi: An attacker can use the UNION operator to combine their query with the one running on the system, this can sometimes yield data from the database and trick it into giving a response.

These attacks are known as *In-band SQLi attacks* in which a malicious query is inputted and the results are shown back through the application, such as the case with the error-based SQLi (the code is sent through the application field and the application shows the information to the user) these methods are responsive and effective..

Content-based SQLi: In this method the attacker tweaks conditions within the query that are true or false (Booleans) and observes how the application's behaviour changes in order to gain more information about the system.

Time-based SQLi: This technique involves the malicious user adding time delays to SQL queries in order to test if a condition is true using the command ("WAITFOR DELAY 00:00:10", where 00:00:10 is an example in the delay time)

Along with the time it takes for the system to respond to try to assess whether a condition is true or not, giving them more information on how the SQL code is formatted.

2.3 Authentication

Authentication is the process of verifying the identity of a user, device or entity before allowing it to gain access to the system, application or network.
This serves as the first line of defence in cyber security , this ensures that only users with the correct identification and authorisation can access sensitive data or resources within the system.

There are different methods of providing authentication from a simple username password login to more complex methods such as biometric scans or multifactor authentication, this can be tedious to the user but is an essential layer of security to help prevent any unauthorised system access from a malicious user.

Authentication is a very important part of any system and organisations that create these systems should always safeguard sensitive information such customer data , intellectual property and financial records , as any leaking of this information can be detrimental to both a companies reputation when it comes to web security and the users that use that system as they entrust their chosen organisation with that data.

2.4 User data privacy rights

With the importance of authentication in regards to allowing user data to be secure and handled properly also comes the responsibility of the organisation to not only secure this data but also handle it properly, with this comes the rights that users have globally.
These rights are globally recognised and enforced, allowing users to hold organisations and businesses accountable for the use of their data, such as the 2018 Facebook scandal in which personal data of over 87 million users was harvested without user consent
Some of the rights granted to users is as follows:

General Data Protection (GDPR)

- This tells organisations what is permitted in their use of personal data, this includes how they should collect, handle and protect personal data.
- **GDPR** states that all users have the right to the protection of any personal data concerning them.
- The data must be processed fairly for specified purposes and on the basis of the consent from the user, they also have the right to access the same data that has been collected on the user.

Health Insurance Portability and Accountability Act (HIPAA)

- This act applies to health insurance companies, Health maintenance organisations (HMOs), company health plans and government based programs that provide health based services such as public healthcare.
- This act protects all information about healthcare staff appointed to a person such as doctors, nurses, and other healthcare providers that may be put on a personal record as well as information about you on the health insurer's database,billing and personal financial information and most other health information that a healthcare provider may be in possession of.

Payment Card Industry Data Security Standard (PCI DSS)

- This differs from the other user rights, where both **GDPR** and **HIPAA** are both acts enforceable by law , the **PCI DSS** is not a law or a legal regulatory requirement, it is a widely accepted set of policies and procedures and is often a part of contractual obligations of organisations that store credit , debit or other card transaction payment types.

- The principles it adheres to is as follows
    1. Build and maintain a secure network and system
    2. Protect cardholder data
    3. Maintain a vulnerability management program
    4. Implement strong access control measures
    5. Regularly monitor and test networks
    6. Maintain an information security policy

3.0 Literary Review

The OWASP Top 10 has been the standard framework for categorising critical web application risks since its 2021 release. This remained unchanged for over four years before being replaced by the 2025 edition (OWASP, 2021; OWASP, 2025). Most of the research in this study was published between 2020 and 2025, so this review uses the 2021 categories as its primary framework, with notes to the 2025 update where they matter.

The review covers six main areas relevant to the research question: broken access control, cryptographic failures and client side data exposure, authentication and two-factor authentication, insecure design and account enumeration, injection and cross-site scripting, and the methodologies used to detect and measure these vulnerabilities under controlled conditions.

3.1 Broken access control

Broken Access Control was ranked the number one risk in OWASP Top 10:2021 and is still ranked first in the 2025 release. Anas, Elgamal and Youssef (2024), writing in the International Journal of Electrical and Computer Engineering, survey the detection landscape and argue that broken access control is unusually difficult to detect automatically, because authorisation logic is application specific rather than pattern matchable.

The survey is descriptive rather than empirical, and the journal is mid tier, so its main use here is as a structured overview of the detection problem rather than as a source of original findings.

However, the empirical evidence supports this position. Huang et al. (2024) found 193 Broken Object Level Authorization vulnerabilities across 25 real world database backed applications, 178 of which had not been previously reported despite the applications being in active use; this was done at the ACM Conference on Computer and Communications Security. Their tool "BolaRay" achieved this with a 21.86% false positive rate, which is non-trivial and worth taking into consideration when interpreting automated detection results. The two sources together suggest that A01 vulnerabilities persist even in mature codebases, and that automated detection has real limitations. This is part of why this study includes manual access control testing rather than relying on tool output alone.

3.2 Authentication and two-factor authentication

Two-factor authentication is cited as a core defence against credential based attacks, and it is one of the controls implemented in the secure application examined in this project. The earliest study considered here, De Cristofaro et al. (2014), is a quantitative usability study of three 2FA technologies (hardware tokens, SMS or email one-time PINs, and authenticator apps) across 219 participants. They concluded that 2FA was generally perceived as usable, and that ease of use, cognitive effort, and trust were the main variables in adoption.

The study has obvious limitations for current research: the sample was drawn from Mechanical Turk, the technologies tested predate widespread smartphone based push authentication, and usability does not equal security effectiveness. It cannot be used to support claims about modern 2FA strength.

Golla et al. (2021), published at USENIX Security, addresses adoption through an industry scale field study at Facebook. The authors tested how messaging design and user experience patterns influenced 2FA opt-in rates, and found that adoption can be substantially improved through notification design rather than additional technical controls. The experiment ran on a live platform with real users, which adds weight to the findings that would not be present in lab based studies, but, like De Cristofaro et al., it is concerned with adoption rather than security outcomes, and the findings are tied to a single corporate environment that may not generalise to smaller deployments such as the one built for this study.

A direct empirical critique of 2FA security is Wang et al. (2024), who developed an automated vulnerability evaluation framework called SE2FA and applied it to 407 production 2FA systems drawn from the Tranco top 10,000 list. They identified systematic weaknesses in the "Remember the Device" cookie mechanisms used to reduce 2FA prompt frequency, and showed that strong 2FA implementations can be undermined by poor session handling. This finding matters for this project because it indicates that simply enabling 2FA is not enough on its own: the surrounding session and cookie logic also has to be hardened. The main caveat is that the paper was an arXiv preprint at the time of writing and has not yet been peer reviewed, so its specific claims should be cited with care.

The three sources together point to the same conclusion. 2FA is necessary but rarely sufficient, and its security depends as much on surrounding implementation choices as on the protocol itself. This is one of the reasons the secure application in this project was designed as a layered system rather than a "normal app plus 2FA" configuration.

3.3 Injection and cross-site scripting

Injection attacks, particularly SQL injection (SQLi) and cross-site scripting (XSS), remain among the most studied web application vulnerabilities, and the project's intentionally vulnerable application provides concrete test cases for both. Disawal and Suman (2023) propose a Web Vulnerability Detection Prevention Methodology that detects and blocks SQLi and XSS attempts using pattern matching and database logging. They report improvements over baseline detection methods, but the paper gives limited detail on its experimental controls and is published in a journal of moderate standing. It is most useful here as an illustration of the typical structure of detection and prevention systems, not as an authoritative source on detection effectiveness.

Tadhani et al. (2024), in Scientific Reports, offer a more rigorous treatment, applying a deep learning approach to XSS and SQLi detection. The authors achieve high reported accuracy on benchmarked datasets, but the work speaks more to defensive tooling than to this study, which is concerned with exploitability rather than detection. It is included here to indicate the current research direction on injection defence and to position this project as complementary, in that it measures exploitability under different security postures rather than developing new detection algorithms.

The defensive SQLi literature is more useful. Sidik, Yutia and Fathiyana (2023) tested parameterised queries across GET, POST, PUT and DELETE operations, and found that parameterisation reliably prevented query manipulation. Choi, Jung and Ko (2025), in MDPI Applied Sciences, extended this analysis with a comparative study of three SQLi defence mechanisms: PHP Data Objects with prepared statements, pattern validation, and an access redirection technique. They concluded that a multi-layered combination is more effective than any single technique. That conclusion matters methodologically here, because it implies that comparing a naive application against a secure one with only one or two defensive layers may underestimate the value of a properly hardened secure posture. Both papers evaluate defences against synthetic payloads in laboratory conditions, and neither captures the behaviour of an attacker iterating against a live system, which is closer to the experimental design adopted in this project.

The closest methodological precedent is Neupane (2025), an arXiv preprint that conducts a penetration test of a PHP/MySQL web application using OWASP ZAP, sqlmap and Nmap, and demonstrates the effectiveness of input sanitisation and prepared statements as countermeasures. The tooling and methodology line up closely with what is proposed here. Its weakness is that it tests a single application rather than comparing multiple security postures, and as a preprint it has not been peer reviewed. It is best treated as a methodological template rather than a citable source for findings, and that single-application limitation is precisely what this project sets out to address.

3.4 Secure coding and testing methodology

The fourth piece of literature considered here concerns how secure code is written and how the resulting applications are tested. Kiashemshaki, Torkamani and Mahmoudi (2025) review secure coding practices across major frameworks and align their analysis with the OWASP Top 10. Alromaizan et al. (2026), in MDPI Computers, synthesise best practices from 35 academic and corporate sources, organising them under authentication, cryptography, input validation, and deployment hardening. Both works are useful for justifying which controls a secure application should implement, but neither offers empirical evidence of how those controls perform under attack. This project addresses that gap experimentally rather than through synthesis.

The methodological literature most relevant here concerns combined static, dynamic and interactive testing. Correa et al. (2021), in Computer Modeling in Engineering and Sciences, proposes a hybrid security assessment methodology that combines white-box and black-box techniques across the secure software development lifecycle. The work is well structured in phases, with the output of each testing stage feeding the next, but it reads as a methodological proposal rather than an empirical evaluation. Mateo Tudela et al. (2020), in MDPI Applied Sciences, addresses that empirical gap directly, combining six security analysis tools (two SAST, two DAST and two IAST) against an OWASP Top 10 benchmark. They report that hybrid combinations meaningfully reduce false positives and improve detection coverage compared with any single tool. The limitation is that the study tests tool combinations against a fixed benchmark rather than across multiple application security postures. The present project inverts that design, holding tools constant and varying the application instead.

3.5 Cryptographic failures and client-side data exposure

Cryptographic failure ranked A02 in the OWASP Top10:2021, covering sensitive data that leaks because protection was not implemented, both at rest or in transit. In modern web applications, the most common form is browser storage misuse: localStorage, sessionStorage, and non HttpOnly cookies are all readable by any JavaScript on the page. Ahmad, Casarin and Calzavara (2023), in ACM Transactions on the Web, ran a large-scale empirical analysis of web storage across the Tranco top 10,000 websites using dynamic taint tracking to classify information flows. They found that third-party scripts routinely access web storage, and that defenses such as Content Security Policy offer all-or-nothing access rather than per-script control. The study is peer reviewed in a top-tier venue and the sample is large. Its main limitation for this project is scope: the authors were studying web tracking, not authentication token theft. The underlying threat model is the same, however, confidential data sitting in client-side storage, readable by scripts outside the application's control. Compagna et al. (2021) on cookie attributes reinforces this: client-side confidentiality depends on where data is stored and which scripts can reach it, and that is precisely what the three-app comparison tests.

3.6 Insecure design and account enumeration

Insecure Design (A04) was introduced in OWASP Top 10:2021 as a category distinct from implementation bugs: it covers weaknesses built into response logic rather than coding errors. Account enumeration is the standard example, because the vulnerability lives in how login, registration and password reset flows respond to valid versus invalid input. Maceiras et al. (2024), in ACM Transactions on the Web, tested 63 popular online services and found that 93.7% were vulnerable to at least one form of account enumeration. The study also included a 318-respondent survey and a 13-participant focus group, a considerably wider evidence base than most work in this area, which relies on automated scanning alone. The main limitation is that the testing was black-box against live services, so the authors could not always determine whether response differences were deliberate design choices or implementation oversights. The 93.7% figure shows that even commercial services routinely fail this category. The methodology used by the authors aligns with WSTG-IDNT-04, which is the test applied in this project.

4.0 Methodology

4.1 Research Design

This research adopts a comparative experimental design to investigate the relationship between a web application's security posture and the exploitability of OWASP Top 10 vulnerabilities. Three functionally equivalent web applications are tested against an identical set of penetration tests. The only meaningful difference between the applications is their security posture. Holding application functionality, technology stack, testing tools, and testing protocol constant allows any observed

difference in exploitability to be attributed to the security posture variable rather than to extraneous factors.

The three applications represent three levels of security posture:

- **Secure application:** built from the ground up with secure coding practices, including two-factor authentication.
- **Naive application:** derived directly from the secure application by stripping out its security features. No vulnerabilities have been deliberately added; it represents a typical implementation in which security has simply not been considered.
- **Vulnerable application:** derived from the naive application by deliberately introducing flaws aligned with the OWASP Top 10 categories.

This three-tier design improves on a binary "secure versus insecure" comparison in two ways. First, it allows the cost of passive insecurity (the naive application, where security was simply absent) to be distinguished from the cost of active insecurity (the vulnerable application, where security is deliberately undermined). Second, because all three applications share an identical codebase, any observed differences across the three are attributable to security posture rather than to differences in functionality, structure, or implementation style.

The core research question this design addresses is: to what extent does a web application's security posture affect the exploitability of OWASP Top 10 vulnerabilities? The dependent variable is exploitability, operationalised as a Pass/Fail/Partial outcome per test plus a CVSS v3.1 severity score for any failure. The independent variable is security posture, operationalised as the three-tier categorical variable described above.

4.2 Framework Selection: OWASP Top 10:2021

The vulnerability classification framework used throughout this study is the OWASP Top 10:2021 (OWASP, 2021), supported by the OWASP Web Security Testing Guide (WSTG) v4.2 (OWASP, 2020) for the technical detail of each test scenario. The 2025 edition of the Top 10 was released in November 2025, partway through this project, but the 2021 edition remains the framework of choice for two reasons.

First, the 2021 edition has been the stable industry reference for over four years, and the body of peer-reviewed literature available to support comparative analysis is built around it. Substantial empirical work exists on individual 2021 categories. Examples include Broken Access Control (Anas, Elgamal & Youssef, 2024; Huang et al., 2024), Authentication and 2FA (Wang et al., 2024; Golla et al., 2021), and hybrid testing methodologies aligned to the 2021 categories (Mateo Tudela et al., 2020). The 2025 edition is too recent to support equivalent analysis.

Second, switching frameworks mid-project would introduce avoidable inconsistency between the test plan, the implementation of the vulnerable application, and the literature drawn on in the analysis chapter. Maintaining the 2021 framework throughout preserves internal consistency.

The WSTG v4.2 is cited as the technical methodology source for individual test scenarios. WSTG v4.2 remains the current stable release of the testing guide, with v5.0 in active development but not yet published.

4.3 Application Architecture

All three applications share the following technology stack:

- **Frontend:** HTML and CSS
- **Backend:** Python with the Flask web framework
- **Database:** MySQL, managed via MySQL Workbench

Each application is derived from the same source codebase. The secure application was developed first, with the naive application produced by removing its security controls and the vulnerable application produced by deliberately introducing flaws into the naive version. This derivation chain matters for the validity of the experimental design: because all three applications share identical functionality and structure, observed differences in test outcomes can be attributed to the security posture variable rather than to incidental variation in features or implementation.

4.3.1 Security controls in the secure application

This secure application uses a number of common and not-so-common security measures. For example, the password hashing function utilises the scrypt

parameters defined within Werkzeug's implementation, and therefore is much more time consuming for an attacker to conduct offline brute-force attacks compared to something like MD5 or SHA-1. It also requires a TOTP code (utilising pyotp) after the submission of the password, which means that having a password stolen is no longer enough for an attacker to gain access to a user's session. The application's database queries are performed via the SQLAlchemy ORM, which automatically parameterises any variables that are passed as input, and is further protected by a custom WAF middleware that enables the second layer of protection against both injection and traversal attacks. Additionally, the session cookie is marked as HttpOnly and has a SameSite attribute set to Lax. All of the responses from the secure application include six HTTP security headers, including Content-Security-Policy and X-Frame-Options. Full implementation detail is provided in Chapter 4.

The naive application is produced by removing the controls listed in section 3.3.1. The vulnerable application is the naive application with deliberately introduced flaws. These include raw string concatenation in SQL queries, unescaped reflected user input, missing access control checks on protected routes, and verbose error handling. Each introduced flaw maps to one or more of the OWASP Top 10 categories listed in section 3.5.

4.4 Testing Environment

Each application is hosted and tested on the laptop of the team member responsible for it. All three team members run Windows, with their respective applications served locally on 127.0.0.1. The secure application runs on port 5000, and the naive and vulnerable applications run on port 5001 on their respective machines. Because the three applications are hosted on three separate machines, the port reuse between the naive and vulnerable applications does not create a conflict.

Testing is conducted entirely against the local instance. Burp Suite Community Edition is used for tests T08 and T10 only. The NIST CVSS v3.1 calculator is used to generate severity scores. Hosting the applications locally rather than on a public network keeps the testing self-contained, with no external systems, networks, or services involved at any stage. This also means test results reflect the behaviour of the application under test rather than network conditions or third-party dependencies.

4.5 Test Selection and Coverage

Following the recommendation by Mateo Tudela et al. (2020) that hybrid testing methodologies provide better OWASP Top 10 coverage than any single tool, this study uses a combined manual and automated testing approach across ten test cases. Each test maps to a specific scenario in the OWASP Web Security Testing Guide v4.2, identified by its stable WSTG identifier.

The ten tests cover six of the ten OWASP Top 10:2021 categories, summarised below:

OWASP 2021 Category	**Tests**	**Coverage Note**
A01: Broken Access Control	T01, T02	Highest-risk category; depth coverage
A02: Cryptographic Failures	T03	Browser storage and transport
A03: Injection	T04, T05	XSS and SQL injection

A04: Insecure Design	T06	New in 2021
A05: Security Misconfiguration	T07, T08	Headers, cookies, and error handling
A07: Identification and Authentication Failures	T09, T10	Direct mapping to research question

The four uncovered categories are explicitly out of scope: A06 (Vulnerable and Outdated Components), A08 (Software and Data Integrity Failures), A09 (Security Logging and Monitoring Failures), and A10 (Server-Side Request Forgery). A06 requires source-level dependency auditing rather than runtime testing. A08 is framework-specific and not relevant to a Flask application of this scope. A09 requires backend log access. A10 depends on the application having URL-fetch features, which is not in scope for the three applications in this study. Acknowledging these as out of scope is preferred to attempting to cover all ten categories with a single superficial test each, an approach that would produce findings of limited methodological weight. The full test plan, including each test's WSTG reference, what it tests, and what its result will show.

4.5 Shared Testing Protocol

This study is conducted as part of a group project in which each team member is responsible for testing the application they personally derived. Because the comparison between the three applications is the central output of the study, the testing process itself must be identical across all three. Otherwise, observed differences could reflect tester behaviour rather than actual security posture.

To address this, all three team members follow a written shared testing protocol agreed in advance of testing. The protocol lays out each step of each test in order, so that every test is conducted in the same way against every application. It specifies identical versions of OWASP ZAP, Burp Suite Community, and the testing browser. It specifies identical scan configurations and tool settings, identical test order (T01

through T10), and an identical evidence-capture standard, with every test producing a screenshot saved with a consistent filename format. CVSS scoring assumptions are also identical, applied using the NIST CVSS v3.1 calculator with the same impact context across all three applications.

The risks associated with each team member testing the application they themselves built, particularly the team member who introduced flaws into the vulnerable application, are addressed in chapter 4 alongside the implementation decisions that shaped the derivation chain.

4.6 Data Collection and Scoring

Every test produces a single row in a shared results spreadsheet. The columns are:

Test ID | OWASP Category | WSTG Reference | Application (Secure / Naive / Vulnerable) | Result (Pass / Fail / Partial) | CVSS Vector String | CVSS Base Score

Severity is scored using the Common Vulnerability Scoring System v3.1 (FIRST.Org, 2019), which provides a standardised numeric score (0.0 to 10.0) and a vector string capturing the attack characteristics that produced the score. CVSS was chosen over alternative scoring approaches because it is the standard used by the National Vulnerability Database, by OWASP itself, and by the wider academic and industrial security community. This means scores produced in this study are interpretable to external readers. The vector strings are recorded alongside the scores so that the impact assumptions made during scoring are transparent and reproducible.

The Pass / Fail / Partial classification provides categorical evidence of whether the relevant security control is present and effective in each application. The CVSS score provides numeric severity for any Fail or Partial outcome, allowing severity-weighted comparisons in the analysis chapter. Evidence screenshots are kept as primary data and are referenced by filename in the spreadsheet.

4.7 Analytical Approach

Once data collection is complete, analysis is structured around the OWASP categories rather than around individual tests, so that findings can be interpreted in the same terms as the framework that produced them. The analysis begins with a results summary: a single table presenting all thirty data points (ten tests across

three applications) alongside their CVSS scores, followed by a descriptive account of the headline counts.

The category-by-category analysis then walks through each of the six covered OWASP categories in turn. For each category, outcomes are compared across the three applications, the differences are explained with reference to what was or was not present in each codebase, and the literature reviewed in chapter 2 is drawn on to support the explanation. This is followed by a discussion of cross-cutting themes that emerge across multiple categories. Examples include the role of framework defaults in blurring the line between the secure and naive applications, the way single flaws can chain into broader exploit paths, and the distribution of severity scores across categories. The chapter closes with a direct answer to the research question, an acknowledgement of limitations, and a short discussion of implications for practitioners and directions for future research.

4.8 Threats to Validity

Several threats to validity apply to this study. Construct validity is limited by the operationalisation of "exploitability" as a Pass/Fail/Partial plus CVSS score; this captures whether an attack succeeds and how severe the consequences are, but does not capture how easily the attack succeeds. Internal validity is supported by the shared codebase derivation chain, but is weakened by the inter-rater bias risk noted in section 3.6. Even with a shared protocol, three testers with three different relationships to their applications cannot achieve perfect consistency. External validity is intentionally limited: this study makes claims about the relationship between security posture and exploitability under the conditions tested, not about production systems in general, and the research question is scoped accordingly. Construct coverage is limited to six of ten OWASP categories, as discussed in section 3.5. Finally, because the test set is finite (ten tests across six categories), absence of evidence of a vulnerability in a particular application is not evidence of absence at the category level. It only means that the specific scenarios tested produced a Pass result.

4.9 Ethical Considerations

All testing is conducted against applications built by the project team, hosted locally on team members' own machines, against synthetic test data only. No third-party

systems, networks, services, or data are involved at any stage of the testing process. Because no external party is affected by any vulnerability identified, no responsible disclosure process is required, and there are no risks to external users or systems.

5.0 Implementation

5.1 Overview

For the purposes of this research, a web-based payroll management system named PaySecure was developed to test its vulnerabilities. It allows the administration of employees, the production of pay slips and control over role-based access to payroll data, making it representative of typical internal business applications which handle large amounts of sensitive personal and financial information.

This employees application allows employees to view their own payslips, their dashboard summary and information in their profiles. Meanwhile, the administrators application allows administrators to view and manage the entire employee roster, generate payslips, view user accounts and monitor all security events.

The frontend of the system is a simple single-page HTML/JS application that is served by a Flask development server. All interactions with the interface are handled by the JavaScript code, which makes requests to the corresponding endpoints in the backend REST API. For organization purposes, the endpoints are divided into three routes: /api/auth for login/register, /api/employee for data that employees of the company have access to, and /api/admin for data and actions that administrators have access to.

All information is stored in a MySQL database split into 4 tables: users, employees, payslips and security_logs. The information is automatically populated on the first run of the software with 6 employees, 3 months of pay history and 2 demo user accounts to help seed the database for you.

The PaySecure implementation evolved over three versions where each version was implemented from scratch using the same codebase and interface but with varying security protection measures. Each version was tested under the same use cases and scenarios and the results were comparable, yet the overhead and limitations due to security decisions differed substantially.

5.2 Common Architecture

So, let's take a closer look at PaySecure - it's actually available in three different versions: secure, naive, and insecure. Despite their differences, they all share the same foundation. For instance, they use Flask as their web framework, rely on Python for the backend, and MySQL for the database, which is accessed through PyMySQL. The database itself is identical across all three versions, with the same schema that includes four tables: users, employees, payslips, and security_logs. Even the routes are the same - you've got /api/auth for login and logout, /api/employee for the employee dashboard and payslips, and /api/admin for administrator functions. From the outside, it's hard to tell them apart - they all run on the same port, serve the same HTML frontend, and have the same login screen. This means that, at first glance, all three versions of PaySecure look and behave in exactly the same way.

What changes between the versions is not the structure but the behaviour inside each route. The secure version applies a full set of security controls before responding to any request. The naive version is what a developer might produce without security knowledge — it keeps the same routes and database but removes the deliberately added protections while retaining whatever Flask provides by default. The insecure version goes further by actively introducing vulnerabilities, replacing safe database queries with raw SQL strings, removing ownership checks, and exposing different error messages depending on what went wrong during login.

5.2.1 Database schema

We’re using four tables to store employee information for the purposes of payroll. Firstly we have the users table to store their username, email, password and the role that they have in the system. The employees table then contains information that is held in the employees company database, such as the employee’s first and second

names, their email address as well as the department and job title that they hold within the company. We then have their salary information, including whether they are employed on a permanent or contract basis and how long they have been employed by the company for. The payslips table stores the employee's gross salary for the particular pay period, any deductions made, their net salary and the period and date over which the pay has been paid. It also includes a flag to show whether the pay has been processed or not. The final table is the security_logs table. This logs security events on the system that have occurred along with information regarding the IP address that the request has been made from, details of the user agent and endpoint that has been used, the actual payload sent in the request and a timestamp against each event. The security logs table is in place in all three versions but is only populated in the secure version where security logging is switched on.

5.2.2 API structure

The API is split into three Flask blueprints: /api/auth for login/log out, 2 factor authentication, and /api/auth/me for the frontend to check if you're logged in on load, /api/employee for the employee dashboard, list of payslips, individual payslip, and employee profile, and /api/admin for adding/deleting amendings/changing employees, generating new payslips, user accounts, and security logs.

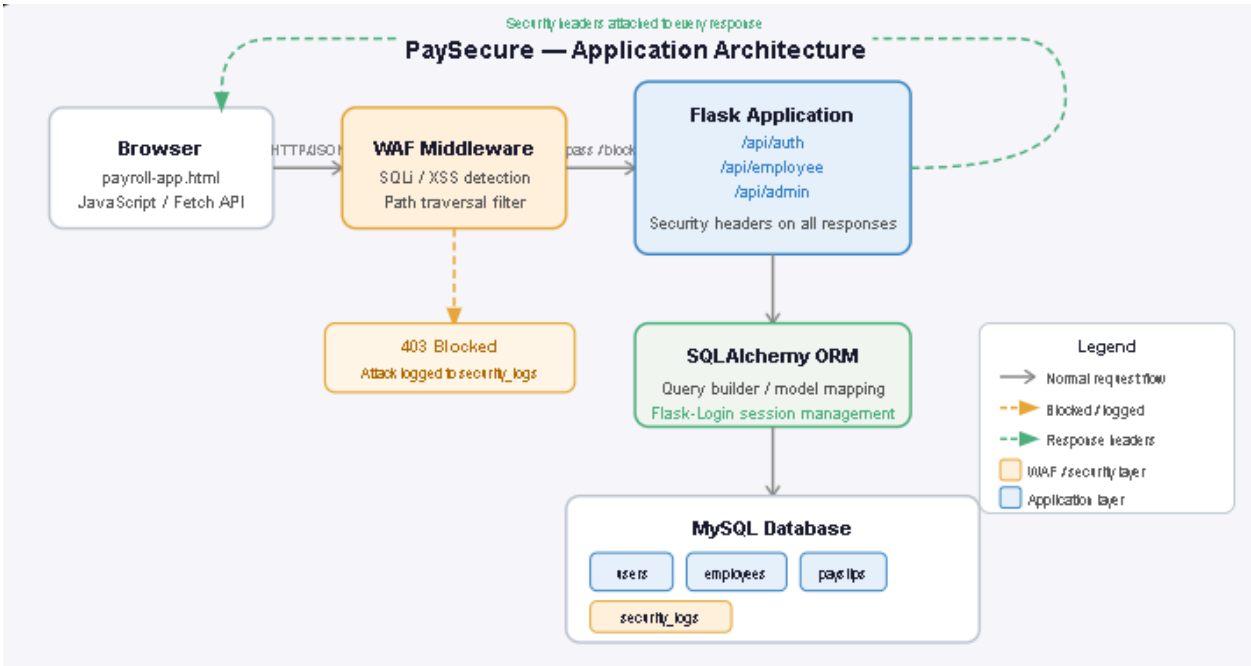


Figure 1 — PaySecure Application Architecture

5.3 Demonstration

5.3.1 Setting Up the Environment

The Setup procedure for all versions of the application is identical. Any differences are highlighted where they occur. Assumed setup is a machine with Python 3.11 or later installed, and a MySQL 8.0 or later server available.

5.3.2 Installing dependencies

All versions include all the necessary packages in the backend directory in the file called requirements.txt. This file includes the following 7 packages: Flask 3.0.3, Flask-SQLAlchemy 3.1.1, Flask-Login 0.6.3, PyMySQL 1.1.0, pyotp 2.9.0, Werkzeug 3.0.3 and cryptography 42.0.8. To install the relevant packages, there is a simple command in the relevant backend folder:

```
pip install -r requirements.txt
```

5.3.3 Database configuration

The application connects to a MySQL database called payroll_db. Before the application is run for the first time, this database needs to be created. The file "payroll_db.sql" in the assets directory of the application contains the schema for this database and should be imported into the system's MySQL server.

sql

```
CREATE DATABASE payroll_db;
```

In the config.py file, are stored the details to connect to the database (user, password, host, database name). The default details are root, Admin1234, on localhost. However, any of the 4 details can be overridden via environment variables (MYSQL_USER, MYSQL_PASSWORD, MYSQL_HOST, MYSQL_DB). For group members replicating the environment, it is probably easier to edit the MYSQL_PASSWORD value in the config.py file.

5.3.4 Starting the server

Remember, to run the full application, you first need to go into the backend directory and run python app.py. This will open a server that listens on port 5000. You can then view the application's frontend at http://localhost:5000 and access the REST API at http://localhost:5000/api. Since we are running everything locally, the server address localhost means that we can access these locations by simply navigating our web browser to the correct address. Running the application for the first time will automatically create the necessary database tables and populate the database with some demo records. This is shown in Figure 4.1. Note how the output confirms that two demo user accounts have been created.

Role	Username	Password
Administrator	admin	Admin:@1234
Employee	johndoe	Employee@1234

5.4 Demonstration of the Application

This section provides details of the PaySecure interface. The following screenshots have been taken from the secure version of the scheme and display the four different screens that a user will encounter during normal use of the service.

5.4.1 Login page

Fig. 2: The first screen presented to the user is the login page, which is straightforward and incorporates space for users' details above the fold.The form is accompanied by the PaySecure branding and a single 'Log in to PaySecure' button.

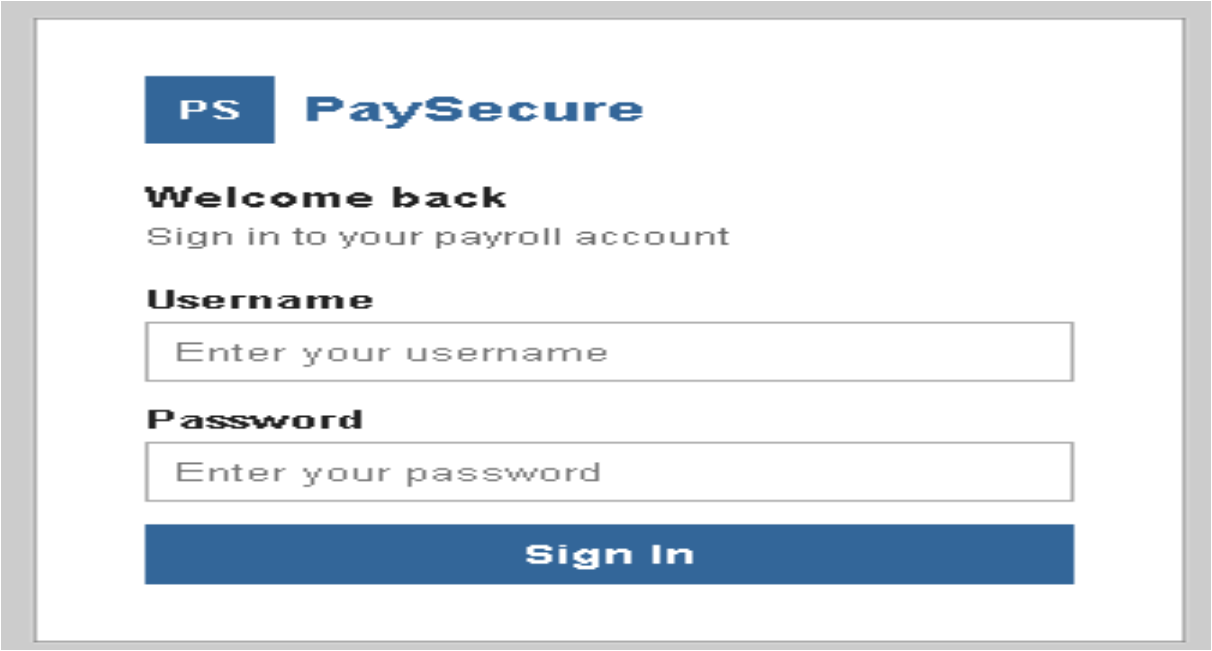


Figure2 (Login page)

5.4.2 Two-factor authentication

Once logged in with the proper username and password the user is greeted with a 2 Factor Authentication login prompt shown in Figure 3. In this example a six digit number is required to move forward with the login process. For demonstration purposes 123456 has been pre-loaded into the figure.

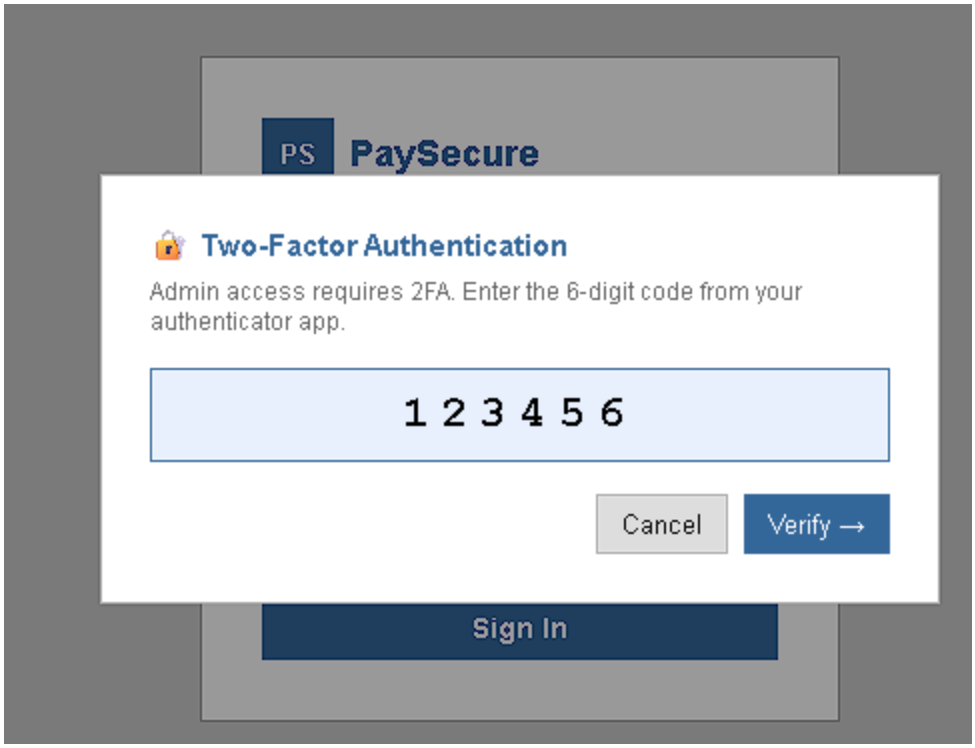


Figure 3 (Two factor Authentication)

5.4.3 Employee dashboard

Employee Dashboard Once logged in and identified as an employee, the system verifies all of the details for that employee and then provides a dashboard-like interface that lists out all of the payslips for that employee. Each listed payslip details the period for the pay, the gross amount of the pay, any deductions made, and the net amount paid.

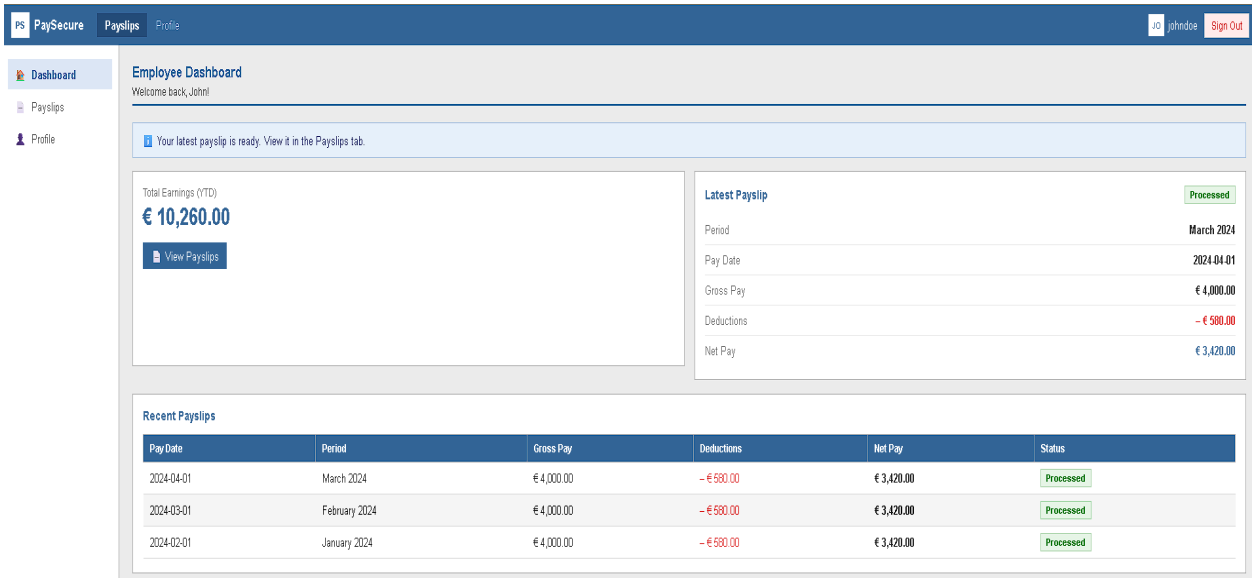


Figure 4(Employee dashboard)

5.4.4 Administrator panel

Users with an administrator account will see a different view after login into the online pay rollover portal. The admin panel shown in Figure 5 presents a full list of all employee records within the system. As shown in the above screenshot, the administrator is able to view payslobs for individual employees as well as manage employee details and also they have the ability to create employee accounts and also see the logs for security in Figure 6

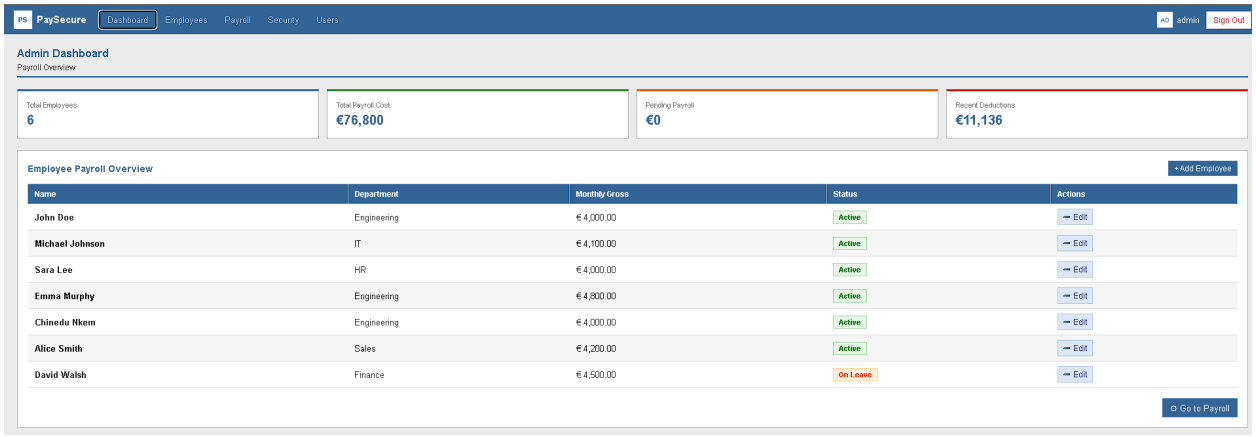


Figure 5(Admin Page)

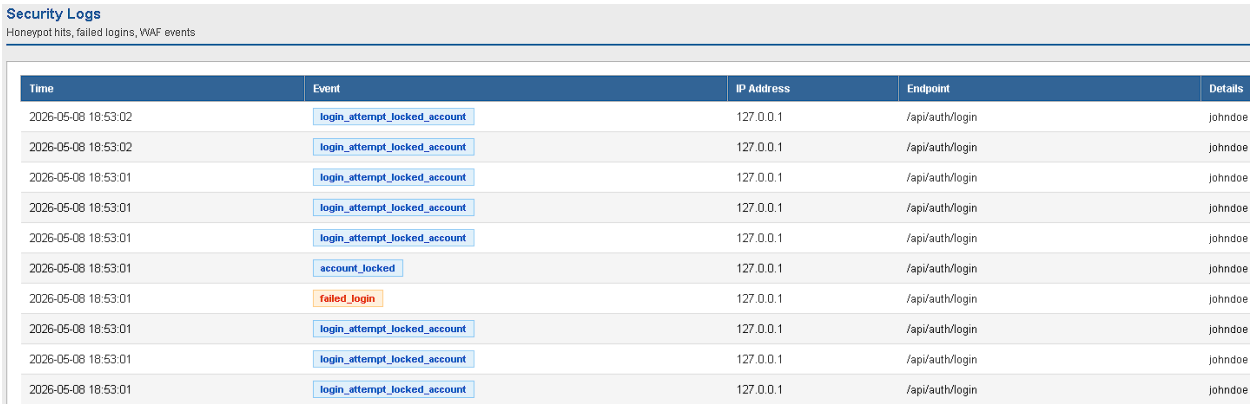


Figure 6 (Security logs page)

5.5 Secure Application

The secure application is the reference implementation for this project and the starting point for the other two versions. Instead of starting with a naive application and then adding security, the naive and vulnerable versions of the application were created by removing or breaking features that are already implemented in the secure version. The following sections describe each of the security controls that were implemented, show what the corresponding code would look like if a developer without a security background were to implement this application, and justify why each control was not likely to be included.

5.5.1 Password Hashing

- A naive developer would likely hash passwords using MD5, as it is a commonly known hashing function and appears in many basic tutorials
- In the MD5 case, precomputed tables of hashes (called rainbow tables) are widely available and can be used by crackers to quickly identify passwords within seconds

```
Python# naive approach
    password_md5 = hashlib.md5(password.encode()).hexdigest()
```

- What we use: Werkzeug's generate_password_hash function which by default uses scrypt, a 'memory-hard' function that makes the cost of brute force much more expensive (Percival and Josefsson, 2016)

```
Python
```

```
    def set_password(self, password):
        self.password_hash = generate_password_hash(password)
```

- Why a naive developer misses this: MD5 is easy to generate and is suitable for checksums, but inexperienced developers do not know to use a different method for passwords because they don't understand the differences between hashing functions and password hashing functions
- Addresses A02:2021 — Cryptographic Failures (OWASP, 2021)

5.5.2 Parameterised Queries

- A naive developer writing a login check might build the SQL query directly from user input using an f-string

python

```
sql = "SELECT * FROM users WHERE username = '%s' AND password = '%s'" %
(username, password_md5)
result = db.session.execute(db.text(sql)).fetchone()
```

- What we use: Flask-SQLAlchemy ORM which correctly parameterises all SQL it generates, so user input gets passed as separate SQL arguments rather than as part of a string

python

```
user = User.query.filter_by(username=username).first()
```

- Why a naive developer misses this: It's the most readable way to write this query, and without knowing what SQL injection is, there is no visible reason to avoid it
- Addresses A03:2021 — Injection (OWASP, 2021)

5.5.3 Generic Error Messages

- A naive login implementation would normally introduce two separate error paths, which is the most natural way to handle these cases

python

```
if not user:
    return jsonify({'message': 'Username not found'}), 401
if not user.check_password(password):
    return jsonify({'message': 'Incorrect password'}), 401
```

- What we use: A single generic response regardless of which check failed

python

```
return jsonify({'success': False, 'message': 'Invalid credentials'}), 401
```

- Why a naive developer misses this: Having separate messages generally makes for a better user experience, and the security implication is not quite obvious enough
- Addresses WSTG-IDNT-04

5.5.4 Account Lockout

- Without this: No limit on login attempts, meaning a password list can be submitted indefinitely
- This change locks an account after 5 failed login attempts for a 15 minute period, resulting in a 429 response

python

```
if user.failed_attempts >= 5:
    user.locked_until = datetime.utcnow() + timedelta(minutes=15)
    return jsonify({'success': False, 'message': 'Account locked. Try again later.'}), 429
```

- Why a naive developer misses this: Flask provides no built-in lockout mechanism. A developer following Flask tutorials would never encounter this unless they went looking for it specifically
- Addresses WSTG-ATHN-03

5.5.5 Two-Factor Authentication

- Without this: Login completes immediately when a correct password is entered, meaning a stolen credential alone is enough for an attacker to log in
- After a correct password, a session flag is set and the user is redirected to a verification step before login_user() is called

python
```
totp = pyotp.TOTP(current_user.totp_secret)
if not totp.verify(code):
    return jsonify({'success': False, 'message': 'Invalid code'}), 401
login_user(current_user)
```

- Why a naive developer misses this: Implementing 2FA requires knowledge of TOTP, a library to work with TOTP, and a separate endpoint to handle the verification. None of these are included by default in a simple Flask application
- Addresses WSTG-ATHN-04 and A07:2021 (OWASP, 2021)

5.5.6 IDOR Protection

- Without this: Any authenticated user can retrieve any payslip by changing the ID in the URL
- An ownership check was added to each payslip request so that the employee's ID is verified against the currently logged in user

python
```
if payslip.employee_id != current_user.employee_id:
    abort(403)
```

- Why a naive developer misses this: @login_required gives the impression the endpoint is protected. The distinction between authenticated and authorised to access a specific record is easy to overlook
- Addresses A01:2021 — Broken Access Control (OWASP, 2021)

5.5.7 Admin Access Control

- Without this: Admin routes protected only by @login_required, so any logged-in user can access admin routes
- A custom admin_required decorator was implemented that checks both if a user is logged in and whether they hold the admin role

python
```
def admin_required(f):
  @wraps(f)
  def decorated_function(*args, **kwargs):
    if not current_user.is_authenticated or not current_user.is_admin():
      abort(403)
    return f(*args, **kwargs)
  return decorated_function
```

- Why a naive developer misses this: Flask-Login's decorator only checks authentication state. Role checking is not included and must be added manually
- Addresses A01:2021 — Broken Access Control (OWASP, 2021)

5.5.8 HTTP Security Headers

- Without this: Flask sends no security-related response headers by default, leaving the browser with no instruction on how to handle scripts, frames or content types
- Six headers are applied to every response via an after_request hook

python
```
response.headers['X-Frame-Options'] = 'DENY'
response.headers['X-Content-Type-Options'] = 'nosniff'
response.headers['Content-Security-Policy'] = (
  "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'"
)
```

- Why a naive developer misses this: These headers are invisible to the end user and the application works identically without them. A developer not familiar with browser security would have no reason to add them
- Addresses WSTG-CONF-07 and A05:2021 (OWASP, 2021)

5.5.9 Session Cookie Security

- Without this: Session cookies are accessible to JavaScript and can be sent cross-site
- HttpOnly and SameSite=Lax flags were set in config.py

python
```
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = 'Lax'
```

- Why a naive developer misses this: Flask doesn't set them by default and nothing in the docs warns you that you need to do this to run a secure application
- Addresses WSTG-SESS-02

5.5.10 Resulting state

To the end user, our new Secure application appears every bit as capable and equally feature rich as our Basic and Professional applications. From the external user perspective, everything looks the same as all three variants are served off the same port, utilize the same basic html frontend, share an identical database schema and have an identical set of routes defined. The only differences occur at the individual routes, where each variant applies a different set of restrictions prior to servicing the query.

5.6 Naive Application

5.6.1 Derivation process

The naive application is not built from scratch. It is derived directly from the secure application by stripping out security features that a developer had to actively add. No vulnerabilities are deliberately introduced. This is what distinguishes the naive

application from the vulnerable application described in section 4.5: the naive application shows what the codebase looks like when a developer has built the functional product without thinking about security, rather than when a developer has actively undermined it.

The derivation began with a fresh copy of the secure application's source tree. Every security control that originated as developer-written code was removed, while framework defaults and application logic were left in place. The result is an application that runs, looks, and behaves identically to the secure version from a user's perspective: same routes, same database schema, same user interface, same business rules. The difference is only visible to an attacker.

5.6.2 What was removed

The following controls were stripped from the naive application. Each is followed by a short justification of why a developer working without security awareness would not have added it in the first place.

- Custom security headers (Content-Security-Policy, X-Frame-Options, Strict-Transport-Security, Referrer-Policy). These have to be set explicitly in Flask, usually via an @app.after_request hook or a library like Flask-Talisman. A naive developer testing the application in a browser sees no functional difference whether these are present or absent, so there is no obvious cue to add them.
- Explicit cookie attributes (Secure, SameSite=Strict). Flask sets HttpOnly on session cookies by default, but Secure and SameSite require explicit configuration via app.config['SESSION_COOKIE_SECURE'] and app.config['SESSION_COOKIE_SAMESITE']. A naive developer would not know these flags exist.
- Server-side role validation on the registration endpoint. The secure application ignores any role field submitted in the registration JSON and assigns the default employee role server-side. The naive version trusts the client-supplied value and assigns whichever role is in the request body. A naive developer treats the registration form as a data-collection step and does not consider that an attacker could send fields the form does not display.

- Two-factor authentication. The TOTP-based 2FA flow, including the QR code enrolment step and the verification step on login, was removed entirely. A naive developer often perceives 2FA as added complexity that frustrates users and skips it on the assumption that a strong password is sufficient.
- Output encoding on the employee table. The secure application populates the employee directory using textContent, which the browser renders as plain text regardless of content. The naive application uses innerHTML, which parses any HTML in the value and executes embedded scripts. A naive developer reaches for innerHTML because most introductory tutorials use it for simple DOM manipulation.
- Custom error handlers. The secure application registers @app.errorhandler callbacks for 404, 500, and uncaught exceptions, returning generic JSON responses with no internal detail. The naive application has no error handlers, so Flask's default behaviour applies. A naive developer assumes the framework's defaults are sensible and never sees the unhandled-exception path during normal development.
- Security event logging. The secure application writes authentication events, role changes, and access denials to a security_logs table. The naive application has no equivalent. A naive developer thinks of logging as a debugging tool, not a security tool, and only logs what they actively need to debug.

5.6.3 What was deliberately kept

Some controls remained in the naive application despite contributing to its security posture. These fall into two categories.

The first is framework defaults. SQLAlchemy parameterises queries by default through its ORM layer, which prevents the most common form of SQL injection without the developer doing anything. Flask sets HttpOnly on session cookies by default, which prevents JavaScript from reading the session token. Removing these would have required the naive developer to actively replace the ORM with raw cursor.execute() calls or override Flask's cookie configuration. Either action is itself a deliberate security-relevant decision and is incompatible with the naive developer model.

The second is application logic that happens to have security implications. The @admin_required decorator on admin routes was kept because without it, the application has no concept of role-restricted areas and the admin pages break for every user. Similarly, the IDOR check on the payslip endpoint (if payslip.employee_id != current_user.employee_id) was kept because without it the application's data model collapses; an employee viewing another employee's payslip is not a security misstep a naive developer would consciously create, but a structural part of how the app distinguishes between users.

The principle is consistent across both categories: security features added by the developer were stripped; framework defaults that protect naive developers without their knowledge were retained, on the grounds that fighting framework defaults would itself be a deliberate decision incompatible with the naive developer model.

5.6.4 Resulting state

The naive application runs on http://127.0.0.1:5001. From a user's perspective it is functionally identical to the secure application: a user can register, log in, view their dashboard, and access their payslips. The same routes resolve to the same handlers, the same database schema is used, and the same HTML interface is served. The only behavioural difference visible to a normal user is the absence of the 2FA enrolment step on registration and the 2FA verification step on login.

The differences are only visible to an attacker, and only become exploitable when actively tested. This is what the naive variant captures: a typical implementation that has been built without security in mind does not announce itself as insecure. It looks finished, functions correctly, and passes a reasonable manual review of its features. The vulnerabilities surface only under structured testing, which is what chapter 5 examines.

5.7 Vulnerable Application

The vulnerable app was configured to broadcast the possible attack surface of sites that are poorly coded and not security minded.

The base of the code for all of the sites was the secure site, each site was individually modified in order to fit the security benchmark that each site needed to reach for accurate and fair testing.

For the vulnerable app, specifically insecure features were added to the code in order to make attacks successful with the purpose of visually demonstrating the capabilities of a malicious user within an insecure system.

Since initially developing the initial site a host of code changes were made to our invulnerable site.

```
async function loadAdminDashboard() {
  const [dash, emps] = await Promise.all([
    api('/api/admin/dashboard'),
    api('/api/admin/employees'),
  ]);
```

Figure 6

T01 - Broken Access control

This vulnerability makes direct calls to the url endpoints shown, it doesn't have a token or role passed in the request. Because the server doesn't do a server-side check either this means any URL pointing at the end points */api/admin/dashboard* or */api/admin/employees* automatically returns admin data.

T02- Insecure Direct Object Reference

```
const data = await api(`/api/admin/employees/${id}`, {
  method: 'PUT',
  body: JSON.stringify(body),
});
```

Figure 7

In this, the employee ID is used in the PUT request and then read from a hidden input field. As we saw in testing a user can change this value in DevTools before submitting and update a completely different employee's table or record.

T03 - Cookie security flags

```
async function api(path, opts={}) {
  const res = await fetch(API + path, {
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    ...opts,
  });
```

In the code the snippet *credentials: 'Include'* tells the browser to attach cookies to every request, this coupled with the fact that the SameSite flag on the server is also missing , this means cookies are sent on cross origin requests by default and CSRF becomes an issue as the frontend does not have a CSRF token mechanism

T05 - Injection

```
# VULNERABLE T06: ADDED FOR SQL INJECTION DEMO
from models import User
sql    = f"SELECT * FROM users WHERE username = '{username}'"
result = db.session.execute(db.text(sql)).fetchone()
```

In this code snippet the username and passwords are sent exactly how they are typed in an input field, there is no validation or encoding, SQL payloads such as *admin' OR '1' = '1'* are not stopped due to the fact that the backend query is not parameterised

Same structure as 4.4 but in reverse
Derivation process - starting point was the secure app then deliberately introduced flaws

What was changed / added bullet point by bullet point with code examples, same as the secure site section - only the most important insecure features you added.

T07 - Missing security Headers

```
<style MDN Reference
  * { margin: 0; padding: 0; box-sizing: border-box; }
  body { font-family: Arial, sans-serif; background: #eeeeee; color: #222; font-size: 14px; }
```

The vulnerable application is written using inline <style> and <script> blocks. A correctly configured Content-Security-Policy header would block inline scripts but because there is not CSP on the server it doesn't.

T08- Verbose error Messages

```
  tbody.innerHTML = data.users.map(u => `
  <tr>
    <td><strong>${esc(u.username)}</strong></td>
    <td>${esc(u.email)}</td>
    <td class="hash">${esc(u.password)}</td>
    <td>${badge(u.role)}</td>
    <td style="font-size:12px;">${esc(u.created_at)}</td>
  </tr>`).join('');
}
```

This code stores password hashes in plaintext, which means if a user uses DevTools in a browser the hash can be broken with the password in plaintext revealed, this is a huge security breach and shouldn't ever occur outside a development environment.
Mapping each flaw to OWASP categories - so "this code snippet is related to A05 in the OWASP top ten , it introduces the flaw because it allows "this" " , – thats just an example of what you can say , but focus on what you added and read 4.4 to see any sections removed from your app that wasn't removed from 4.4 and why etc etc

5.8 Implementation challenges

5.8.1 Testing protocol finalised late

The agreed protocol for the tests (test IDs, OWASP methodology, results recording) wasn't finalised until after we had started testing the secure version. As a result, there was some risk of inter-rater bias, and the three of us could have interpreted the same test differently. A framework was established prior to testing the insecure and naive versions of the code, however, to ensure the same steps, expected outcomes and evidence were recorded consistently across the three tests. This issue did arise during testing of the secure version, but in the end it didn't affect our

results. Nonetheless, it is an issue that should be addressed prior to beginning a testing project.

5.8.2 Framework defaults blurring the security boundary

As we were developing, we discovered that there were some protections already built into the frameworks we were using, and that weren't something we had to add ourselves. Flask has support for HttpOnly cookies built-in, and SQLAlchemy has built-in protection against SQL injection because of the way it handles queries. This made it harder to draw a clean line between what the secure version added on purpose and what was just there by default. The solution was to only mark something as a security control if it actually took extra work to implement.

5.8.3 Development errors and use of AI assistance

The secure version had a couple of technical issues that needed to be corrected during the development process and required debugging in multiple locations for conflicts in session management and in the database schema. AI tools were used as a development aid to identify issues and suggest corrections. This really helped, as at some point there were a lot of errors that I had trouble locating and it assisted me in resolving those issues . This also really helped me with time management because it would have taken double the time to locate and solve.

6.0 Analysis

6.1 Overview

This study asked, how does a web application's security posture affect its exploitability of the OWASP Top 10:2021 vulnerabilities? To investigate this question, 10 penetration tests, outlined in the OWASP Web Security Testing Guide v4.2, were conducted on three functionally identical applications written with the Flask web framework to handle employee payroll actions. One web application

included explicit security measures in its source code, the second did not, and the third was filled with vulnerabilities on purpose. Each penetration test can yield multiple results per tested web application, and this study saw thirty total results from the penetration tests on the three payroll web applications, with each failure ranked using the CVSS v3.1 severity score. The three payroll web applications all utilised the same codebase, same technologies, and same routes; therefore, the thirty failures can be analysed by OWASP vulnerability categories, then as a whole, and then used to answer the research question.

6.2 Results summary

Test ID	OWASP Category	WSTG Reference	App	Result	CVSS Vector	CVSS Score	Notes
T01	A01 - Broken Access Control	WSTG-ATHZ-02	Secure	Pass	N/A	0	401 unauthenticated, 403 employee, CSP blocked role injection attempt
T01	A01 - Broken Access Control	WSTG-ATHZ-02	Naive	Fail			
T01	A01 - Broken Access Control	WSTG-ATHZ-02	Vulnerable	Fail	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:N	8.5	
T02	A01 - Broken Access Control	WSTG-ATHZ-04	Secure	Pass	N/A	0	CSP blocked fetch to admin endpoint; admin_required enforces role check server side
T02	A01 - Broken Access Control	WSTG-ATHZ-04	Naive	Fail			
T02	A01 - Broken Access Control	WSTG-ATHZ-04	Vulnerable	Fail	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N	6.5	
T03	A02 - Cryptographic Failures	WSTG-CRYP-03 / WSTG-CLNT-12	Secure	Pass	N/A	0	HttpOnly and SameSite=Lax set; session cookie not readable via JS
T03	A02 - Cryptographic Failures	WSTG-CRYP-03 / WSTG-CLNT-12	Naive	Partial			
T03	A02 - Cryptographic Failures	WSTG-CRYP-03 / WSTG-CLNT-12	Vulnerable	Fail	AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:L	9.6	
T04	A03 - Injection	WSTG-INPV-01	Secure	Pass	N/A	0	WAF and CSP blocked XSS payload before reaching route
T04	A03 - Injection	WSTG-INPV-01	Naive	Fail			
T04	A03 - Injection	WSTG-INPV-01	Vulnerable	Fail	AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:L	8.9	
T05	A03 - Injection	WSTG-INPV-05	Secure	Pass	N/A	0	ORM parameterises all queries; SQLi payload returned no result
T05	A03 - Injection	WSTG-INPV-05	Naive	Pass			
T05	A03 - Injection	WSTG-INPV-05	Vulnerable	Fail	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H	10	
T06	A04 - Insecure Design	WSTG-IDNT-04	Secure	Pass	N/A	0	Generic invalid credentials message returned regardless of failure reason
T06	A04 - Insecure Design	WSTG-IDNT-04	Naive	Fail			
T06	A04 - Insecure Design	WSTG-IDNT-04	Vulnerable	Fail	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	
T07	A05 - Security Misconfiguration	WSTG-CONF-07 / WSTG-SESS-02	Secure	Pass	N/A	0	X-Frame-Options: DENY prevents iframe embedding
T07	A05 - Security Misconfiguration	WSTG-CONF-07 / WSTG-SESS-02	Naive	Fail			
T07	A05 - Security Misconfiguration	WSTG-CONF-07 / WSTG-SESS-02	Vulnerable	Fail	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N	6.1	
T08	A05 - Security Misconfiguration	WSTG-ERRH-01	Secure	Pass		0	No stack trace exposed; generic 400/404 returned on malformed requests
T08	A05 - Security Misconfiguration	WSTG-ERRH-01	Naive				
T08	A05 - Security Misconfiguration	WSTG-ERRH-01	Vulnerable	Fail			
T09	A07 - Identification & Authentication Failures	WSTG-ATHN-04	Secure	Pass	N/A	0	2FA required after password step; login blocked without valid TOTP code
T09	A07 - Identification & Authentication Failures	WSTG-ATHN-04	Naive				
T09	A07 - Identification & Authentication Failures	WSTG-ATHN-04	Vulnerable	Fail			
T10	A07 - Identification & Authentication Failures	WSTG-SESS-07	Secure	Pass	N/A	0	Account locked after 5 failed attempts; 429 returned for subsequent tries
T10	A07 - Identification & Authentication Failures	WSTG-SESS-07	Naive				
T10	A07 - Identification & Authentication Failures	WSTG-SESS-07	Vulnerable	Fail			

Figure 8

The results show a clear difference between the three application versions. The secure application performed the strongest overall, passing all recorded tests. This shows that the security controls implemented in the secure version were effective against the tested OWASP scenarios. These included access control checks,

two-factor authentication, secure cookie settings, generic error messages, parameterised database handling, a web application firewall, and security headers.

Secure	10	0	0	0	10	0.0%		0.00	0.00
Naive	1	1	5	0	7	71.4%	0.00	0.00	0.00
Vulnerable	0	0	10	0	10	100.0%	5.49	10.00	54.9(

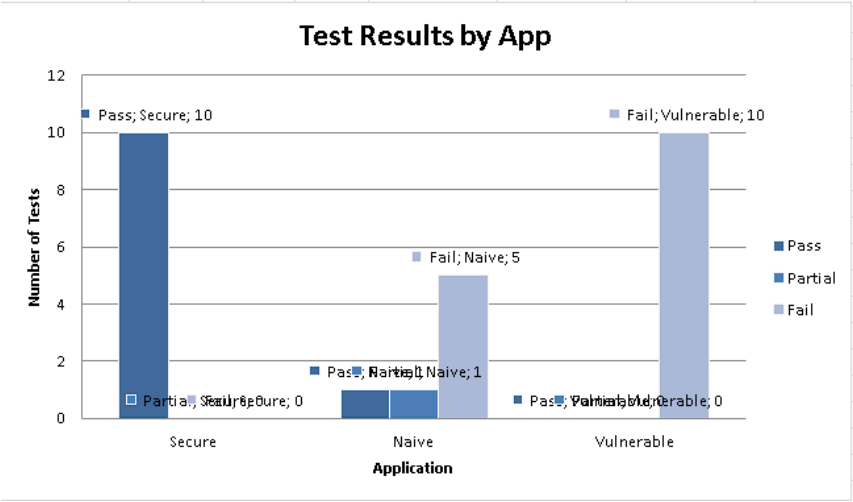


Figure 9

The vulnerable application performed the weakest overall. It failed all of the recorded tests, with several failures receiving high or even critical CVSS scores. The most serious results appeared in SQL injection, cross-site scripting, cryptographic failures, and broken access control. This was expected because the vulnerable version had deliberately insecure behaviour introduced into it, such as missing authorisation checks, insecure cookie settings, raw SQL handling, and weak error handling.

Per-OWASP-Category Summary

OWASP Category	App	Pass	Partial	Fail	Failure Rate	Average CVSS (Fail/Partial)
A01 - Broken Access Control	**Secure**	2	0	0	0.0%	
A01 - Broken Access Control	**Naive**	0	0	2	100.0%	0.00
A01 - Broken Access Control	**Vulnerable**	0	0	2	100.0%	7.50
A02 - Cryptographic Failures	**Secure**	1	0	0	0.0%	
A02 - Cryptographic Failures	**Naive**	0	1	0	0.0%	0.00
A02 - Cryptographic Failures	**Vulnerable**	0	0	1	100.0%	9.60
A03 - Injection	**Secure**	2	0	0	0.0%	
A03 - Injection	**Naive**	1	0	1	50.0%	0.00
A03 - Injection	**Vulnerable**	0	0	2	100.0%	9.45
A04 - Insecure Design	**Secure**	1	0	0	0.0%	
A04 - Insecure Design	**Naive**	0	0	1	100.0%	0.00
A04 - Insecure Design	**Vulnerable**	0	0	1	100.0%	5.30
A05 - Security Misconfiguration	**Secure**	2	0	0	0.0%	
A05 - Security Misconfiguration	**Naive**	0	0	1	100.0%	0.00
A05 - Security Misconfiguration	**Vulnerable**	0	0	2	100.0%	3.05
A07 - Identification & Authentication Failures	**Secure**	2	0	0	0.0%	
A07 - Identification & Authentication Failures	**Naive**	0	0	0		
A07 - Identification & Authentication Failures	**Vulnerable**	0	0	2	100.0%	0.00

Figure 9

The naive application sat between the secure and vulnerable applications. In some tests it failed because important security features had been removed, such as stronger authentication controls and explicit security headers. However, in other areas it performed better than the vulnerable version because some framework-level protections still remained. This is important because it shows that modern frameworks can reduce certain risks even when a developer has not deliberately implemented strong security controls. However, relying on framework defaults alone was not enough to provide complete protection.

6.3 Analysis by OWASP Category

6.3.1 A01 — Broken Access Control

6.3.1.1 Secure Site

T01 — Vertical Privilege Escalation (WSTG-ATHZ-02)

This vector was also tested by attempting to register a new account with role: admin passed via the request. The browser blocked the request entirely, displaying: "Refused to connect because it violates the document's Content Security Policy." The connect-src 'self' directive in the application's CSP header prevents the page from making any requests outside of its own origin. This attack was therefore stopped before it reached the server.

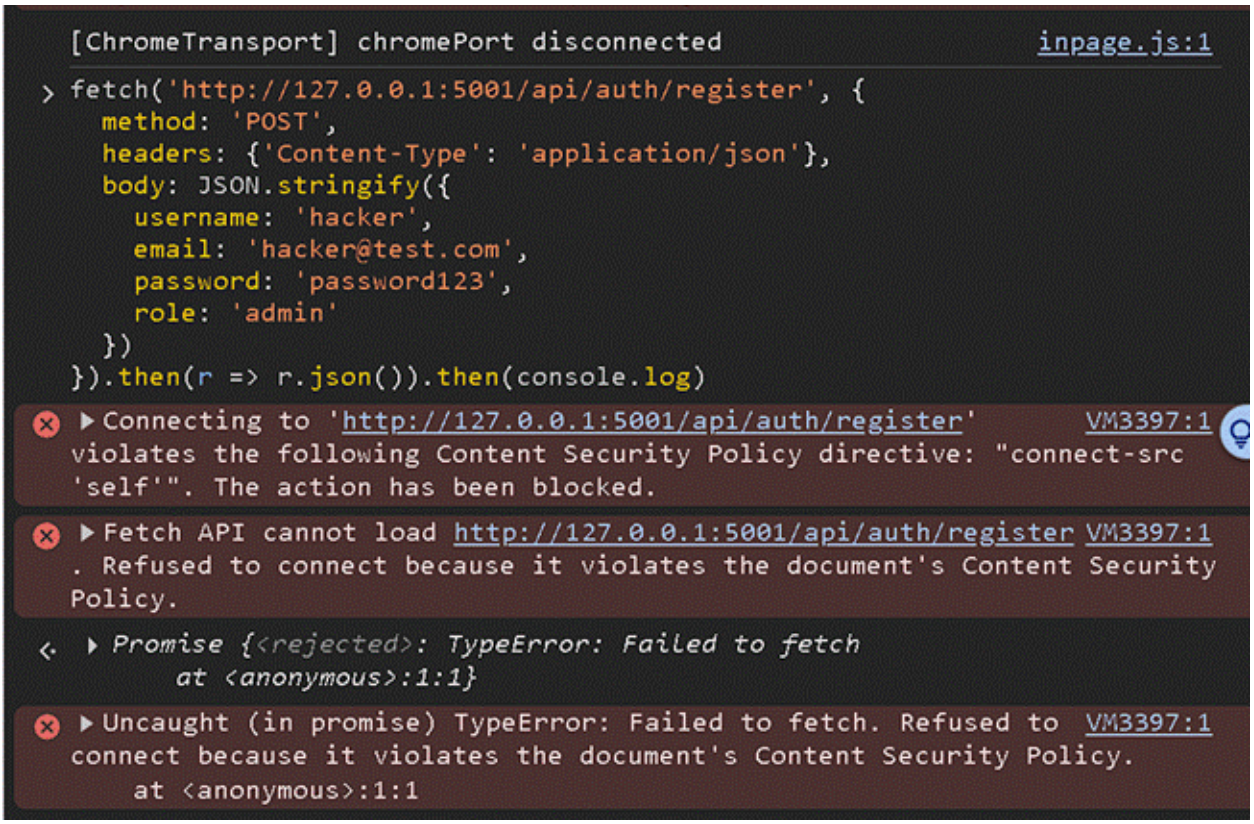


Figure 10

What makes this work in the code:

@login_required — returns 401 if no valid session exists before the route is reached

admin_required — custom decorator in admin.py that checks both is_authenticated and is_admin(); returns 403 if either check fails

connect-src 'self' — CSP header set in after_request hook in app.py; blocks any fetch or XHR to a different origin, stopping client-side role injection before it leaves the browser

T02 — Unauthorised Function Level Access (WSTG-ATHZ-04)

This test attempted to access the admin employees endpoint directly through the browser console using a fetch command. The attempt was first made logged in as an admin, and then as a regular employee.

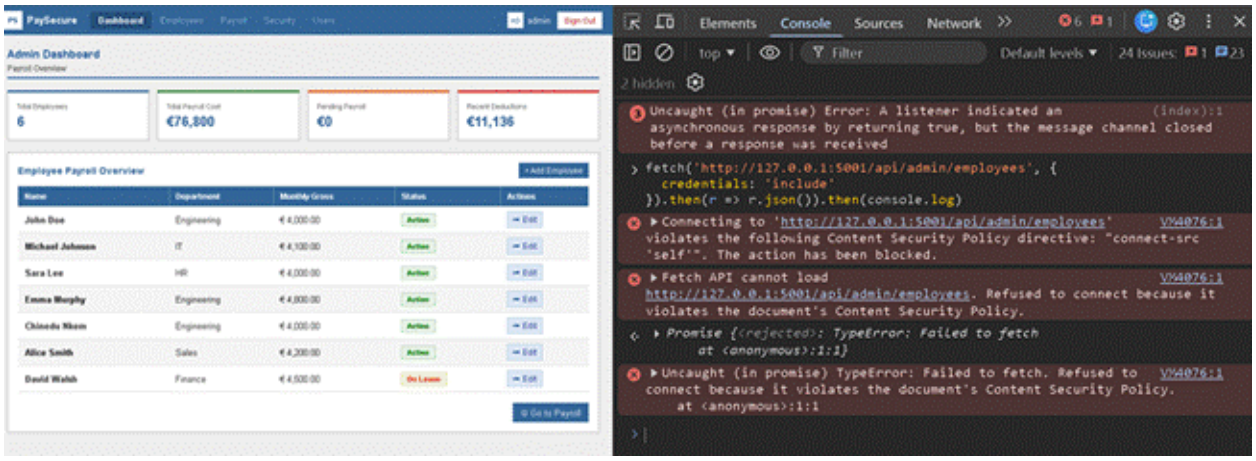


Figure 11

In both cases the request never reached the server. Running the fetch while logged in as admin produced the same CSP error as running it while logged in as johndoe — "Refused to connect because it violates the document's Content Security Policy." The connect-src 'self' directive treats 127.0.0.1:5001 as a different origin from the page's own origin, blocking the outbound connection at the browser level before any credentials are sent.

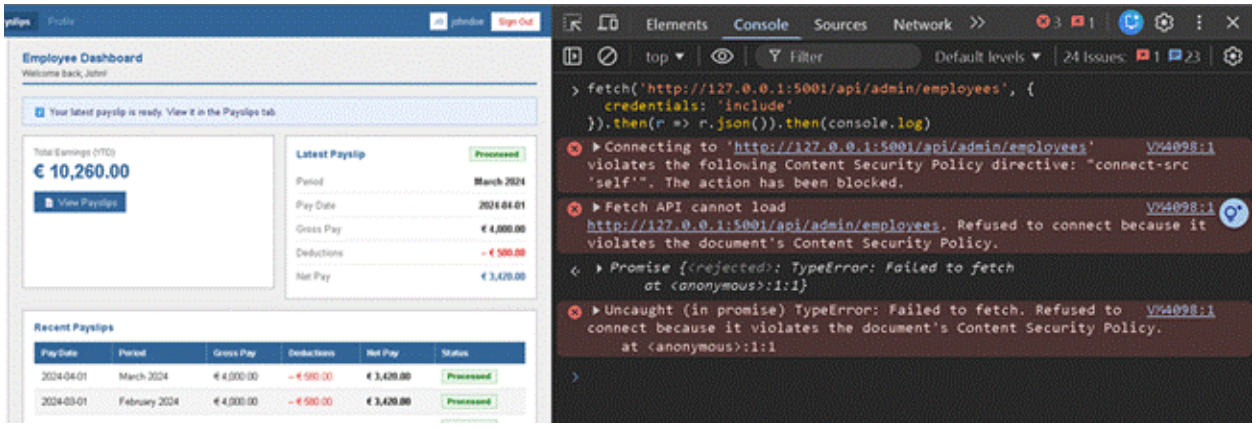


Figure 12

The result confirms that even if an attacker is authenticated, they cannot use console-based fetch calls to probe the API from within the application's own pages.

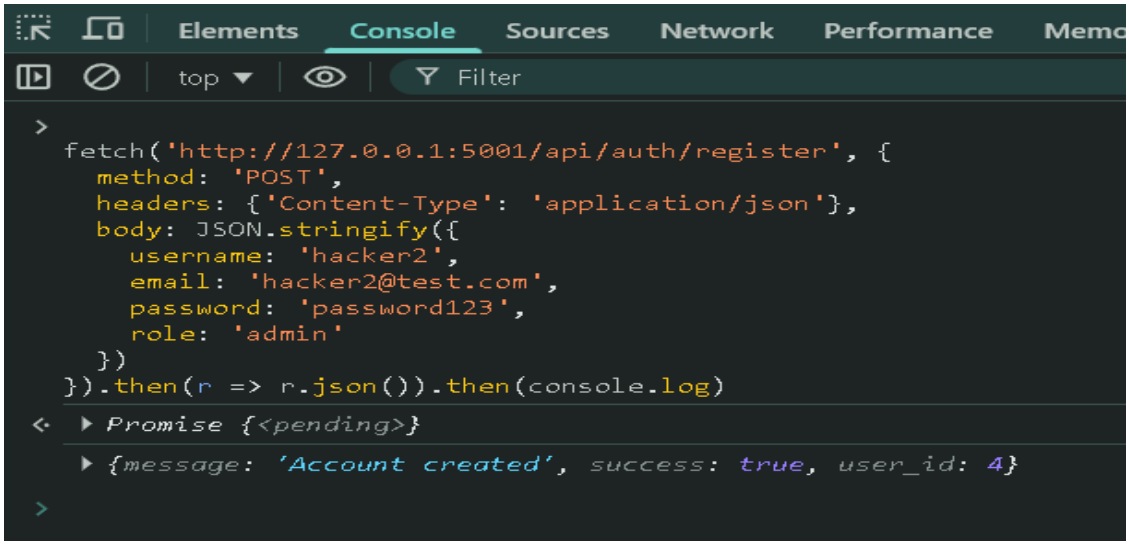
What makes this work in the code:

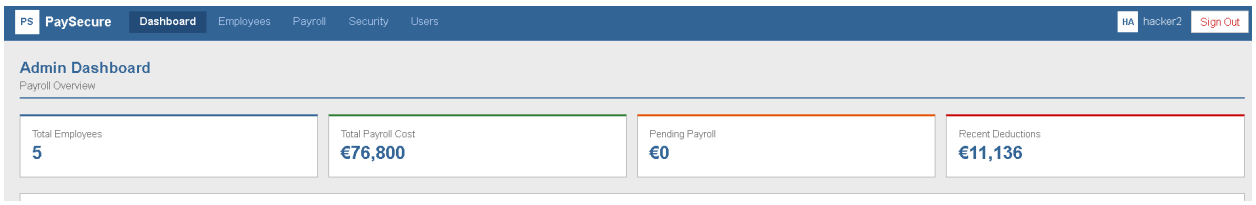
```
connect-src 'self'  — applied in the after_request hook in app.py; any
fetch or XHR to a different origin (including 127.0.0.1:5001 from a page
served on 5000) is blocked by the browser before the request is sent

admin_required  — even if the CSP were removed, an employee account
would receive a 403 before any data was returned
```

6.3.1.2 Naive Site

T01 — Vertical Privilege Escalation (WSTG-ATHZ-02) — Fail

The naive application failed T01. CVSS v3.1 score: 9.1 (Critical), vector AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N. A POST to /api/auth/register with the body {"username":"hacker2","role":"admin",...} was accepted, and logging in as hacker2 then loaded the full Admin Dashboard.



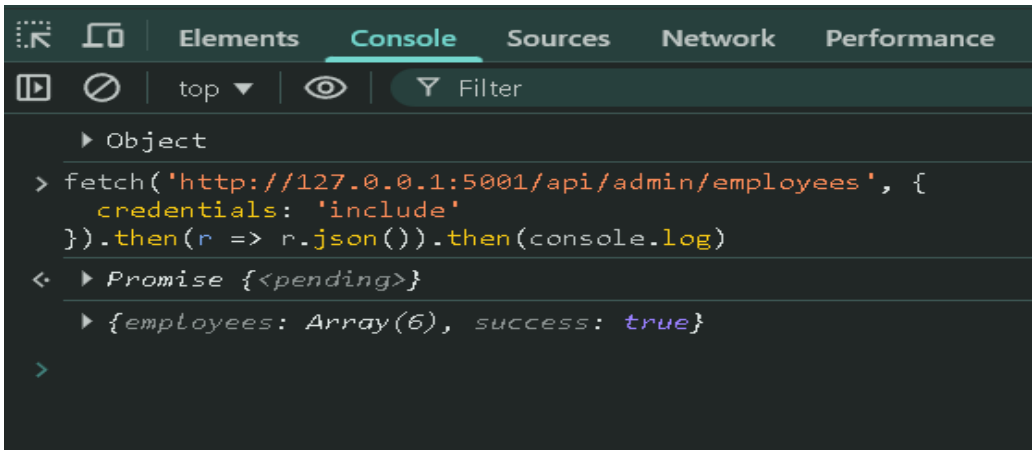


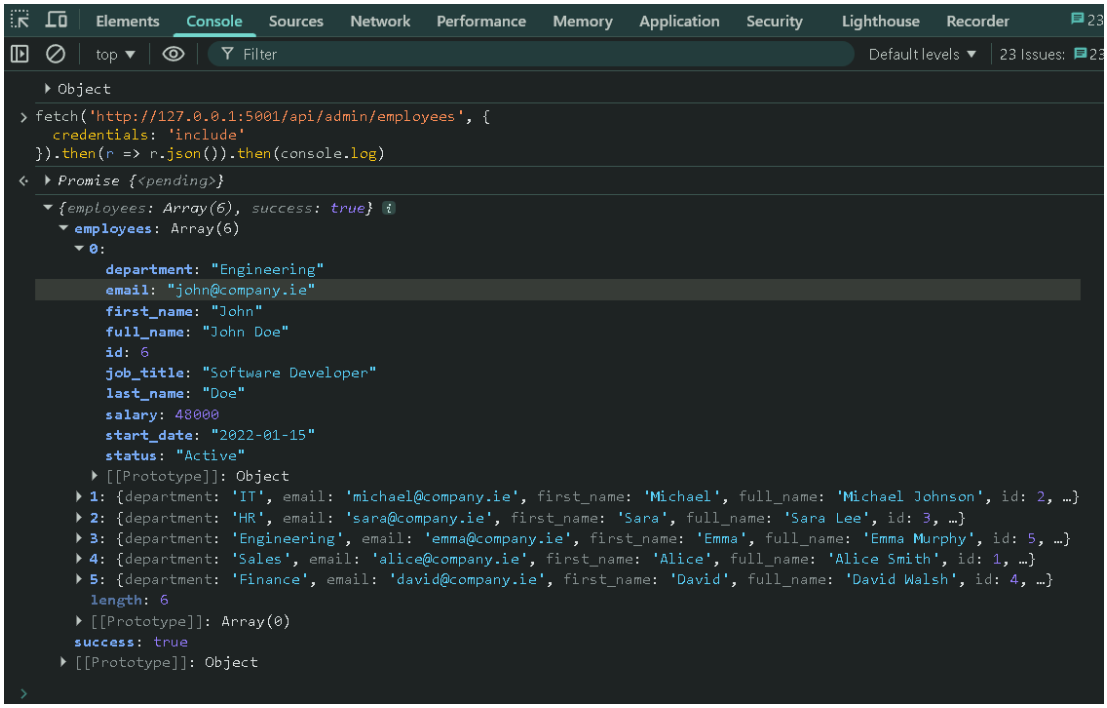
This is a mass-assignment vulnerability. The secure variant ignores any client-supplied role field on registration and assigns the default employee role server-side. The naive variant trusts whichever role the request body contains and writes it directly to the database. The @admin_required decorator on admin routes

was kept on the naive site (section 5.6.2), so the route-level control is still in place, but the value it checks has been set by the attacker at registration. An unauthenticated attacker can take admin access in a single request, with no interaction from any other user, which is what produces the critical score.

T02 — Unauthorised Function Level Access (WSTG-ATHZ-04) — Fail

The naive application failed T02. CVSS v3.1 score: 6.5 (Medium), vector AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N. A fetch against /api/admin/employees returned a 200 response with the full employees array — names, salaries, departments and emails for all six employees.





The endpoint should require admin privilege, and on the naive site the @admin_required decorator is still applied to it. The root cause is the same as in T01: the role check passes because the requesting account holds the admin role. The test itself is independent, since this is a function-level access check rather than a privilege-escalation primitive. The score is lower than T01's because a session is required (PR:L) and the data is read-only (I:N), with high confidentiality impact from full disclosure of employee records.

6.3.1.3 Vulnerable Site

T01 - Vertical privilege escalation (WSTG-ATHZ-02) -Fail

This test checks whether a basic user level account can exploit a web applications system in order to escalate their own privileges to a higher level or to gain access to/create an account with those privileges.

This test was performed on the insecure site and confirmed that a user level account was able to create an admin account through console in DevTools

Figure 13

By navigating directly to the console and inputting
the application returned a full list of employee records, this included information such as names, start dates, roles , financial information email and more, all belonging to another user or administrators.
This is a dangerous exploit that could potentially lock out users from their own systems through abuse of admin privileges.

This result shows that the insecure app's security posture performed no server-side validation of user privilege before returning sensitive data.

The attack required no additional knowledge other than knowledge of JS so the barrier to exploit is low.

T02: Unauthorised Function Level Access (WSTG-ATHZ-04)- Fail

Much like the previous test this test also involves the console window in DevTools.

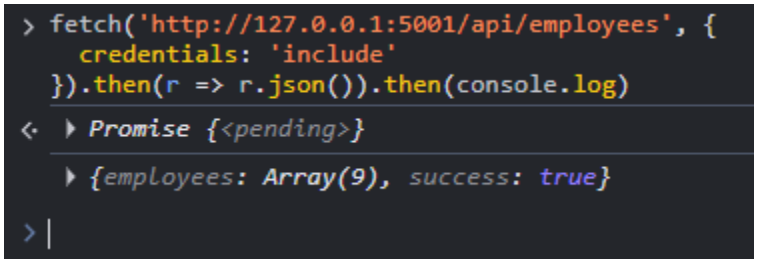


Figure 15

Figure 16

From the code being executed it was able to grab all of the information pertaining to a table in which employee information is being kept, this is a huge security breach as not only does it display personal information but also roles and monetary information that could be detrimental to any users that have their data within the database.

6.3.2 A02 — Cryptographic Failures

6.3.2.1 Secure Site

T03 — Cookie Security and Session Handling (WSTG-CRYP-03 / WSTG-CLNT-12)

After logging in successfully, we examined the session cookie issued by the application using the browser's own tools, inspecting it in detail to see what attributes were assigned. We found three key attributes to be: HttpOnly, SameSite, and Secure.

The HttpOnly flag is set to true on the secure application. This flag tells the browser to not allow the cookie to be accessed by JavaScript (i.e. document.cookie will not be able to read it). This provides an additional layer of protection as even if an attacker were to perform an XSS attack on the site the script would not be able to read or exfiltrate the session token. Without HttpOnly an attacker would only need to perform one XSS vulnerability in order to steal an active session and impersonate the logged in user.

We set SameSite to Lax. This setting controls whether or not a browser sends the session cookie for a given domain with requests made to other sites. By setting it to Lax, we prevent the cookie from being submitted in cross-site requests that come from sources such as embedded images or forms on another site. This functionality can be used to prevent cross-site request forgery attacks. An attacker would not be able to trick an already logged in user to submit a legitimate request on the attacking site.

This Secure flag is not set, which is fine for our local development environment, but would prevent this cookie from being sent over unencrypted HTTP connections. This flag is enabled automatically for our production environment via the COOKIE_SECURE environment variable.

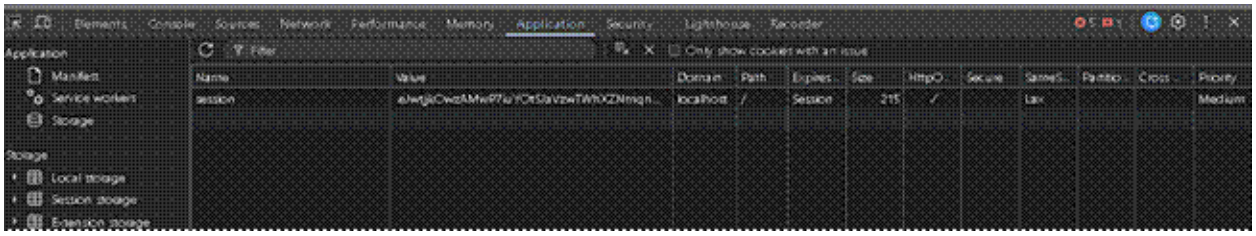


Figure 17

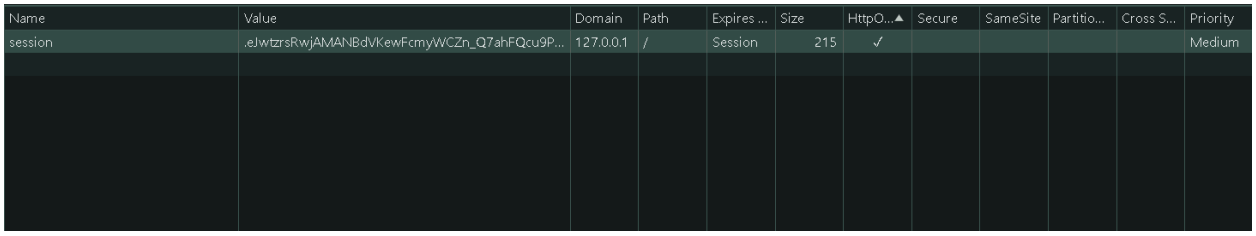
What makes this work in the code:

```
SESSION_COOKIE_HTTPONLY = True  — set in config.py; prevents JavaScript from accessing the session cookie

SESSION_COOKIE_SAMESITE = 'Lax'  — set in config.py; prevents the cookie being sent on cross-site requests
```

6.3.2.2 Naive Site

T03 – Cookie Security and Session Handling (WSTG-CRYP-03 / WSTG-CLNT-12) – Partial

The naive application produced a Partial result on T03. CVSS v3.1 score: 3.7 (Low), vector AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N. Inspection of the session cookie in DevTools confirmed HttpOnly was set, but SameSite and Secure were both absent.



Name	Value	Domain	Path	Expires ...	Size	HttpO...	Secure	SameSite	Partitio...	Cross S...	Priority
session	.eJwtzrsRwjAMANBdVKewFcmyWCZn_Q7ahFQcu9P...	127.0.0.1	/	Session	215	✓					Medium

HttpOnly is set automatically by Flask-Login's defaults, which is why the naive site retains it without the developer having added it. SameSite and Secure require explicit configuration via app.config, and a developer unaware of these flags would have no prompt to set them (section 5.6.2). The cookie is therefore protected against script-based theft but exposed to CSRF and to interception over HTTP. The result is classified Partial rather than Fail because one of the three relevant flags is in place, even if it is in place by inheritance rather than by design.

6.3.2.3 Vulnerable Site

T03 — Cryptographic Failures (WSTG-CLNT-12 / WSTG-SESS-02) Cookie Security Flags-Fail

This test involves the inspection of the application's session cookie via browser DevTools.

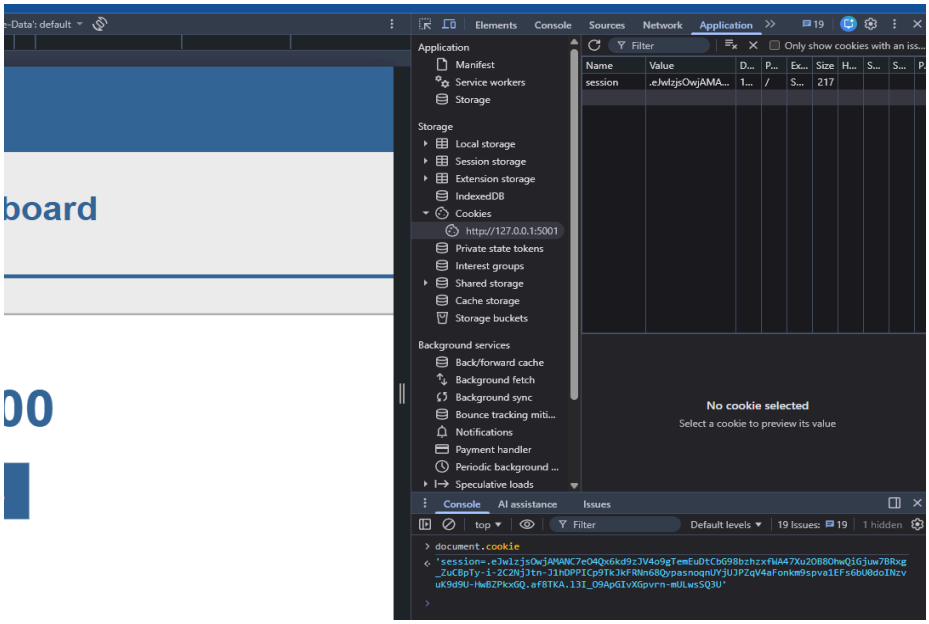


Figure 18

Inspection with DevTools confirmed that HttpOnly, SameSite and Secure attributes were absent (unchecked).

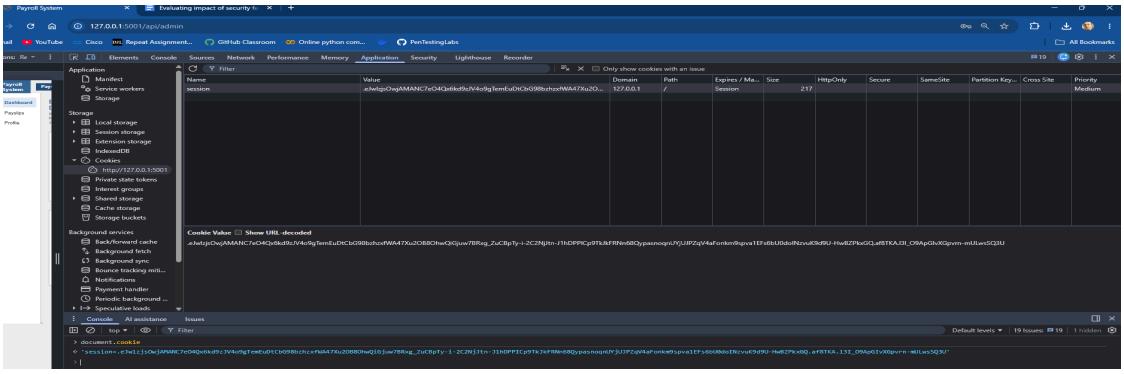
The document.cookie command was then entered in the console section of DevTools, upon execution of that command the session cookie's value was returned , confirming that it was fully accessible to JavaScript.

This test demonstrates that the applications cryptographic and session management posture enables further exploits.

HttpOnly being disabled means that any successful script injection can steal a users session tokens, in addition to this the SameSite option being unchecked exposes users to Cross-Site Request Forgery (CSRF).

This is a huge vulnerability as will be shown in a further test where XSS is confirmed and cookie theft is demonstrated on the insecure site.

6.3.3 A03 — Injection



6.3.3.1 Secure Site

T04 — Cross-Site Scripting (WSTG-INPV-01)

This test assessed the possibility of injecting reflected and/or stored XSS using a script payload in a single form input. The attack string used was :

```
<img src=x onerror=alert('XSS')>
```

On the secure version of the site, the same payload was inserted into the First Name field of the Add New Employee form. The result was a "Request blocked by security filter" response. The request was blocked at the form level and was never submitted to the server logic or database.

As our analysis shows, the payload was captured at the first layer of processing, before it could reach the server logic or database. This speaks directly to the first research question: rather than being constrained to post-processing or frontend validation, the protection afforded by the WAF middleware operates as proactive server-side filtering, intercepting the request before it is handled by any application logic.

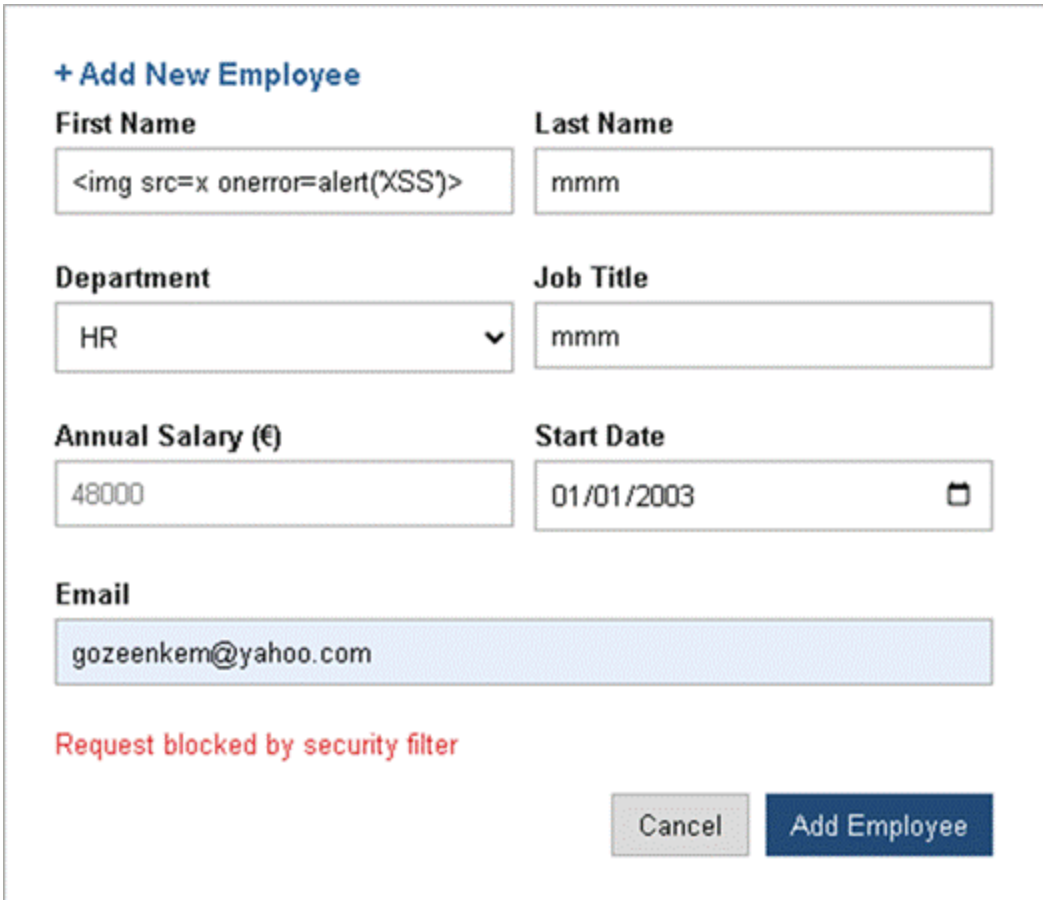


Figure 19

What makes this work in the code:

```
WAF in waf.py  — registered in app.py before any blueprints; runs on
every incoming request and checks all input fields against regex
patterns; detects HTML tags, script patterns, and event handlers like
onerror= and returns a 400 immediately
```

T05 — SQL Injection (WSTG-INPV-05)

The aim of this test was to see if an attacker could tamper with database queries by using user input to get around security checks or steal data. The payload that was used was:

```
' UNION SELECT username, password_hash, 3, 4, 5 FROM users--
```

On the secure version, the payload was entered into the username field on the login page. The application returned "Request blocked by security filter" immediately. The

WAF detected the UNION SELECT pattern and rejected the request before it reached the database layer.

This result confirms what we found out earlier - the WAF middleware doesn't rely on the application's logic to detect bad input. So, the protection it offers is consistent and can't be easily gotten around by targeting a different entry point or creating a request that skips the frontend. This means that no matter how someone tries to sneak in malicious input, the WAF will still be able to catch it and prevent it from causing harm. It's not like some other security measures that can be bypassed if you know the right trick - this one is solid and dependable.

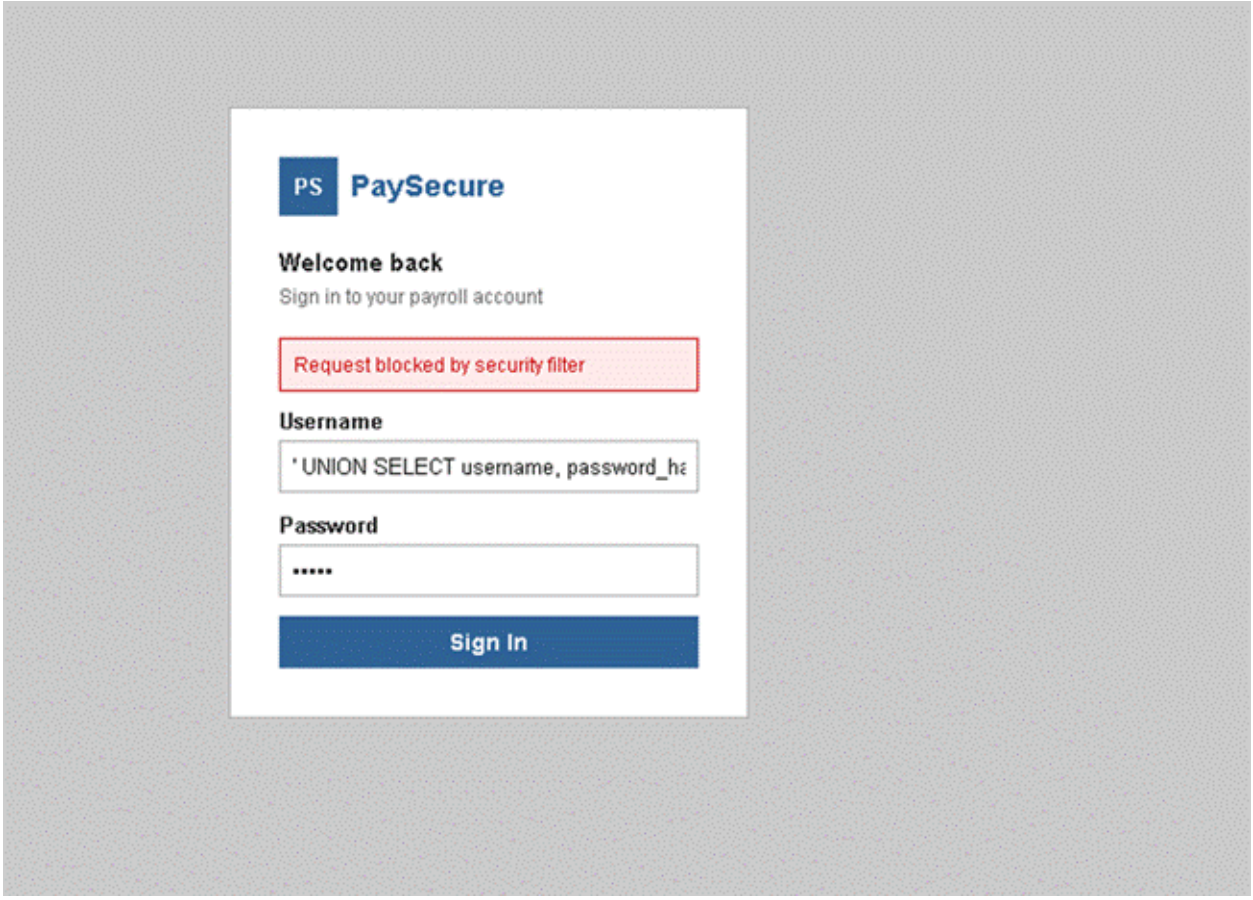


Figure 20

What makes this work in the code:

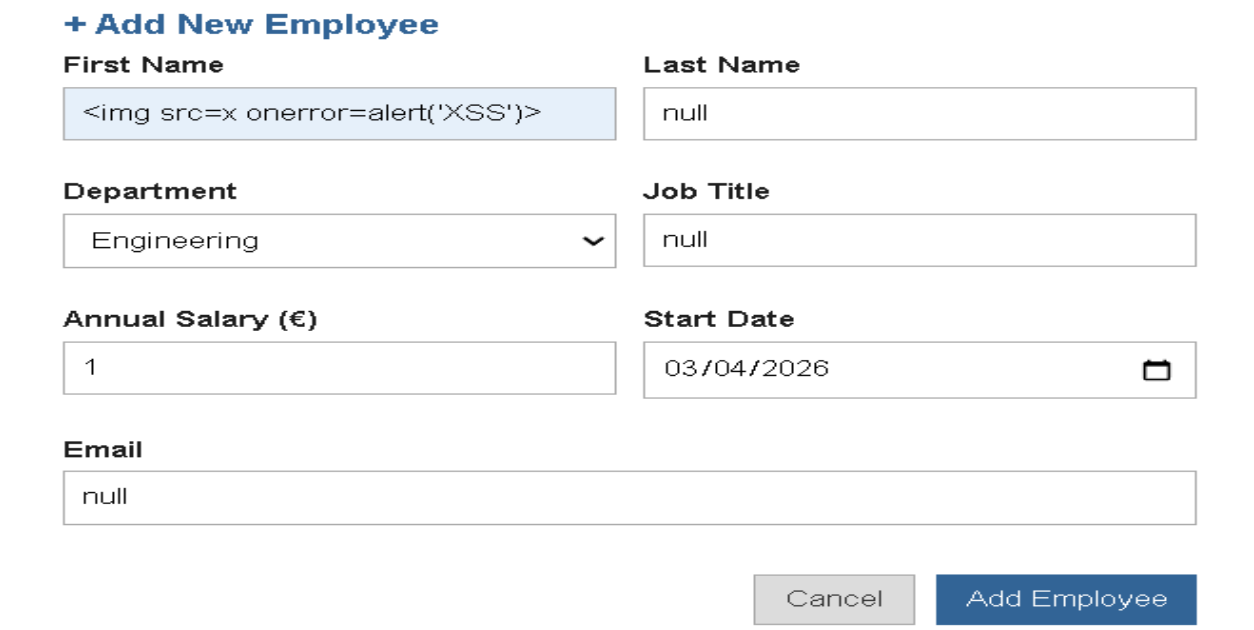
WAF in waf.py — includes regex patterns that detect UNION, SELECT, and quote-based manipulation

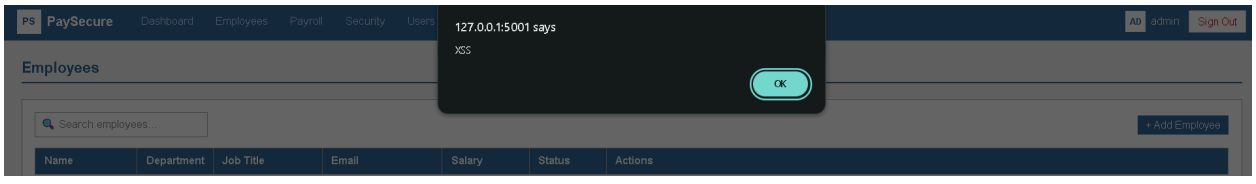
SQLAlchemy ORM — never concatenates raw user input into SQL strings; even a crafted payload is treated as a literal string value rather than executable SQL

6.3.3.2 Naive Site

T04 — Cross-Site Scripting (WSTG-INPV-01) — Fail

The naive application failed T04. CVSS v3.1 score: 5.4 (Medium), vector AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N. The payload <img src=x onerror=alert('XSS')> was submitted into the First Name field of the Add Employee form. The payload was stored, and the alert('XSS') dialog fired when the Employees page was rendered.

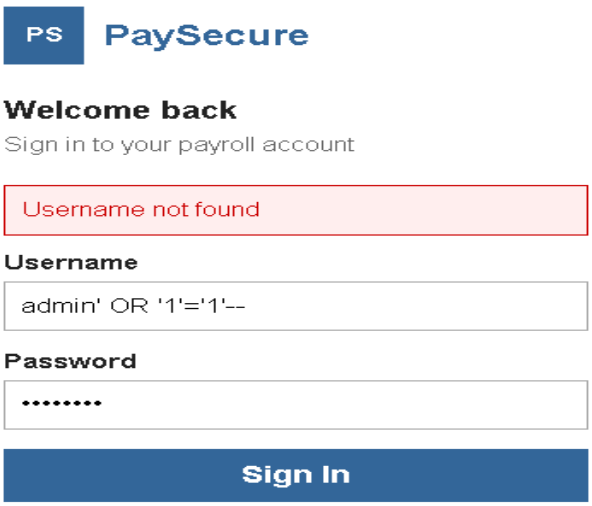




The naive site populates the employee table using innerHTML, which parses embedded HTML and executes handlers like onerror. The secure variant uses textContent, which renders the same value as plain text. Neither variant filters input. The secure site's defence is a WAF middleware that the naive site lacks, but the immediate cause here is the rendering choice. Confidentiality and Integrity are scored Low rather than High because the HttpOnly flag from T03 closes the most direct session-theft path, limiting the script to in-page actions and authenticated requests on the existing session.

T05 — SQL Injection (WSTG-INPV-05) — Pass

The naive application passed T05. The payload admin' OR '1'='1'-- entered into the username field returned "Username not found", and the login was rejected.



The pass is the work of the SQLAlchemy ORM, which parameterises every query at the driver level by default. The injection payload is treated as a literal username, and because no user has that literal name, the lookup fails. To make this test fail on the naive site the developer would have had to actively replace the ORM with raw query construction, which is the change introduced into the vulnerable site. This is not evidence of a deliberately implemented defence. It is a framework default that the naive developer inherited without knowing they needed it, and shows the boundary problem set out in section 5.8.2.

6.3.3.3 Vulnerable Site

T04 — Cross-Site Scripting (WSTG-INPV-01)-Fail

This test involves the attempted execution of code into an input field in order to assess input validation.

The injection of the code "*<img src=x onerror=alert('XSS')>* " was performed into the "Add employee" input field in the employee Dashboard.

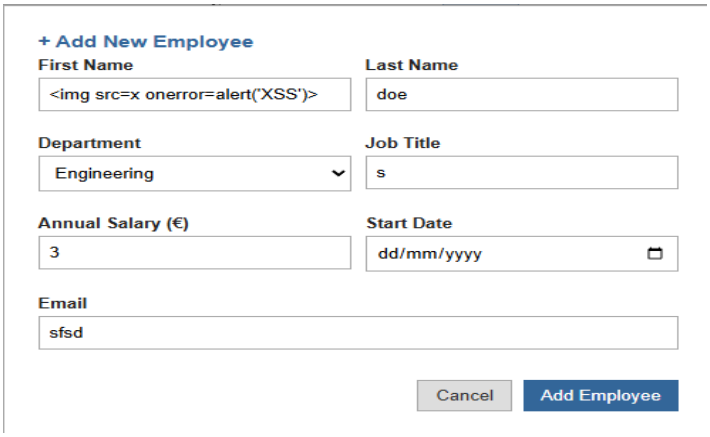


Figure 21

The form was then submitted , Once this was entered a pop up appeared returning the message 'XSS' showing that shows the code was successfully executed, this is vulnerable on the site because of the previously mentioned sameSite and httpsOnly cookie settings that are disabled.

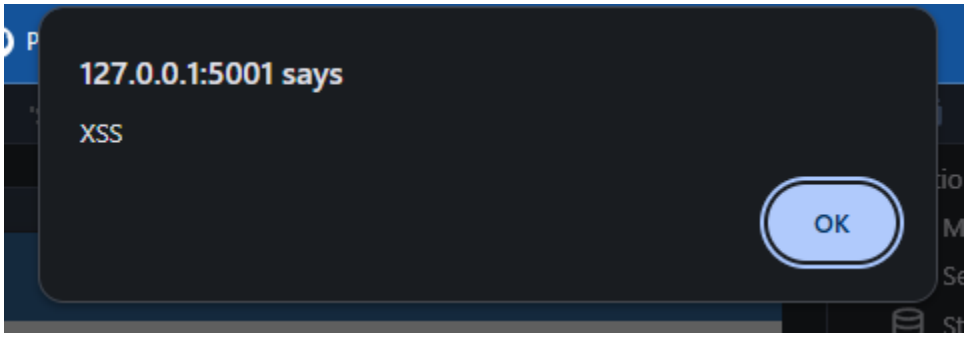


Figure 22

This result confirms that the application has no output encoding or input validation, this means any content supplied by the user and entered into fields is automatically trusted despite the possibility that they could input code such as in this test

T05 — SQL Injection (WSTG-INPV-05) — Advanced Payload
This tests whether an app can be exploited using SQL queries, the cause of this vulnerability is non parameterised queries.
Malicious users can pull any user data from a given database or table , exposing user information for possible future attacks.

Since the SQL code can be inputted into any field (such as search fields or text fields) while logging into the site the code :

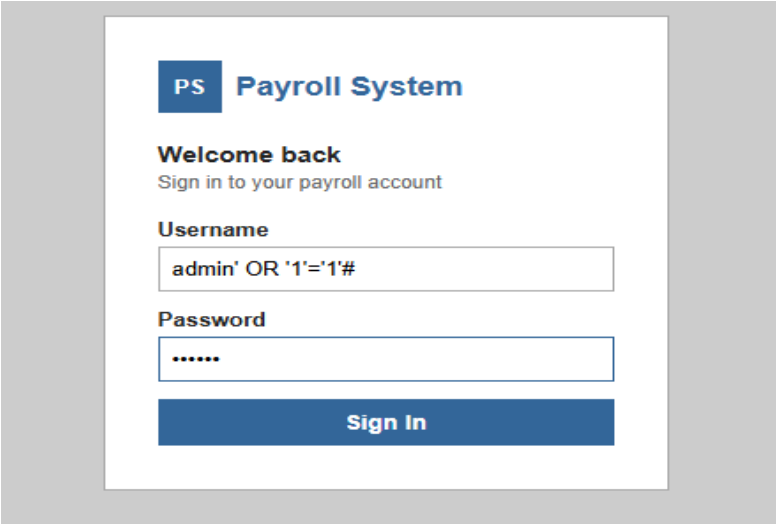


Figure 23
Admin' OR '1'='1'# into the field it was able to log me in. When the code is inputted and submitted in the form , the system takes it as executable code, the line of code input pulls the password from the database and submits the form, which effectively inserts the password in for the user, completing the login.

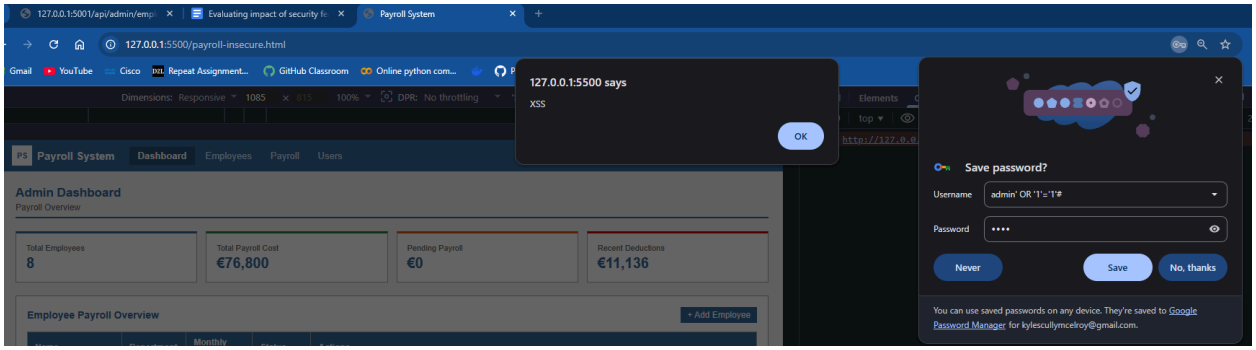


Figure 24

In the form above you can see that the system takes the SQL code as a valid username, as the code pulls the information and inputs it , despite the entered password being incorrect, this is a huge breach for a site holding user information and the insecure site fails at this test.

6.3.4 A04 — Insecure Design

6.3.4.1 Secure Site

T06 — Account Enumeration (WSTG-IDNT-04)

The method used to exploit this vulnerability was to look for a difference in the size of the response from the application, depending on whether a username was valid or not. Even if the error messages looked the same, a tiny difference of just one byte in the response size could be enough for an attacker to figure out which accounts were real. To test this, two login attempts were made with incorrect passwords, both of which got a 401 response, but the key was to check if the response sizes were different.

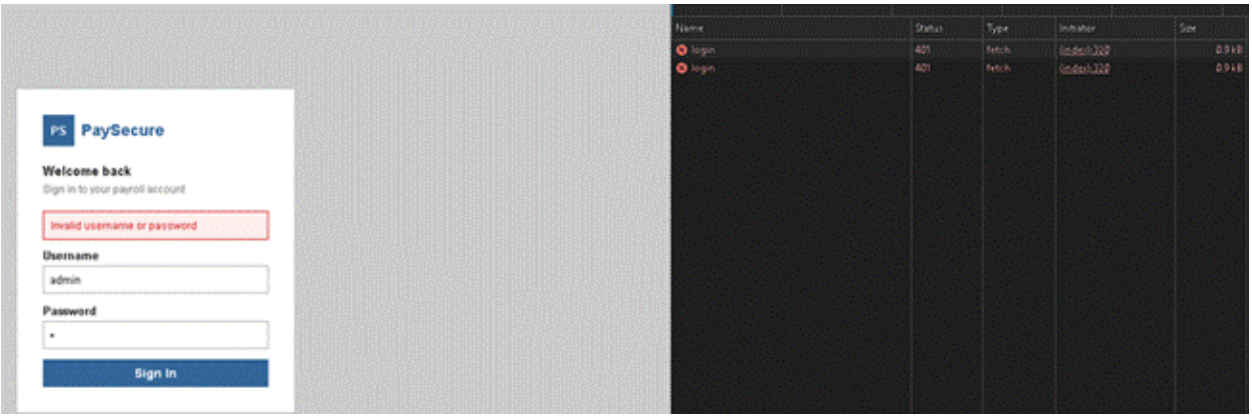


Figure 25

The responses were exactly the same, with no difference in status code, size, or error message. This is a problem because if the sizes were different, it would be easy for an attacker to figure out which usernames are valid. They could send lots of login attempts with different usernames and see which ones get a different response. This would give them a list of targets to try and guess the passwords for. It's like giving them a roadmap to launch a brute force attack. By making all the responses the same, it's much harder for them to get that information.

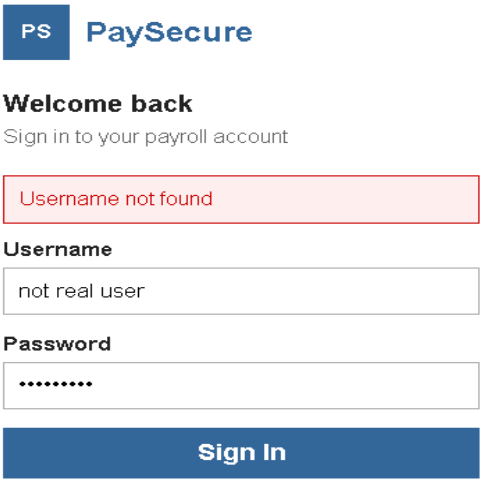
What makes this work in the code:

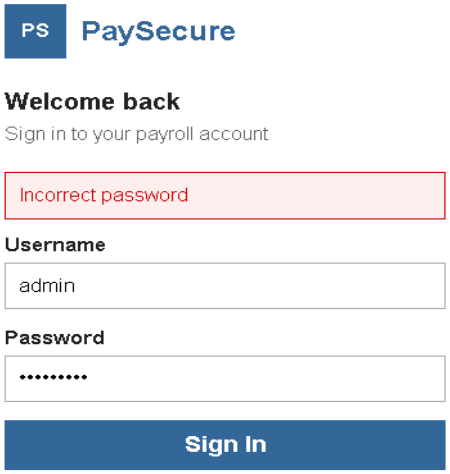
```
routes/auth.py  — always returns {"success": false, "message": "Invalid username or password"} whether the username exists or not, making it impossible to distinguish between the two cases from the outside
```

6.3.4.2 Naive Site

T06 – Account Enumeration (WSTG-IDNT-04) – Fail

The naive application failed T06. CVSS v3.1 score: 5.3 (Medium), vector AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N. Two failed login attempts produced different responses. A non-existent username returned "Username not found", while an incorrect password against a valid username returned "Incorrect password".





The secure variant collapses both branches into a single Invalid credentials message specifically to prevent enumeration. The naive variant uses the most natural login flow: check whether the user exists, and if not, say so; otherwise check the password. The two messages are independently sensible to a developer thinking about usability, and they let an attacker confirm valid usernames. Maceiras et al. (2024) found 93.7% of commercial services tested were vulnerable to this category, which supports the OWASP framing of A04 as a design-level concern rather than an implementation bug.

6.3.4.3 Vulnerable Site

T06 — Account Enumeration (WSTG-IDNT-04) — Response Size Discrepancy

This test involves the information that an attacker can gain from a given system using the error messages the attacker receives from said system.
An example of this is in a login screen, while a secure app may give general error information such as “invalid username or password” an insecure app will say either “incorrect username” or “Incorrect password”.

The reason that this is insecure is because this can allow a user to build a list of usernames and other account information that can allow them to more accurately execute an attack.

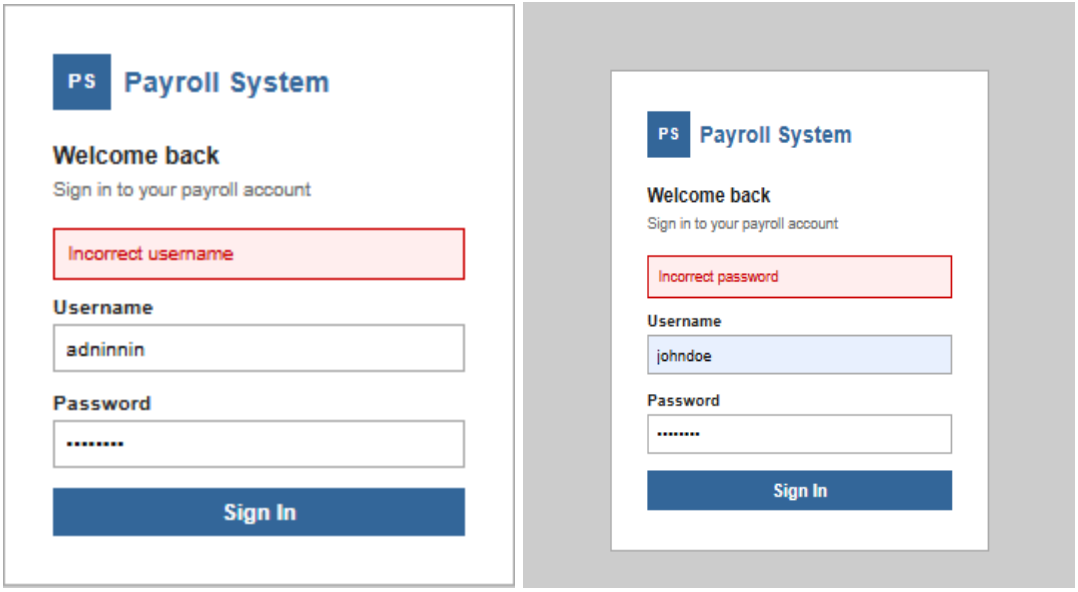


Figure 26

6.3.5 A05 — Security Misconfiguration

6.3.5.1 Secure Site

T07 — Security Headers and Clickjacking (WSTG-CONF-07)

I checked the response headers directly using the browser DevTools by clicking on the dashboard api request and looking at the Headers tab. This confirmed the presence of all security headers and their proper configuration on every response from the secure application.

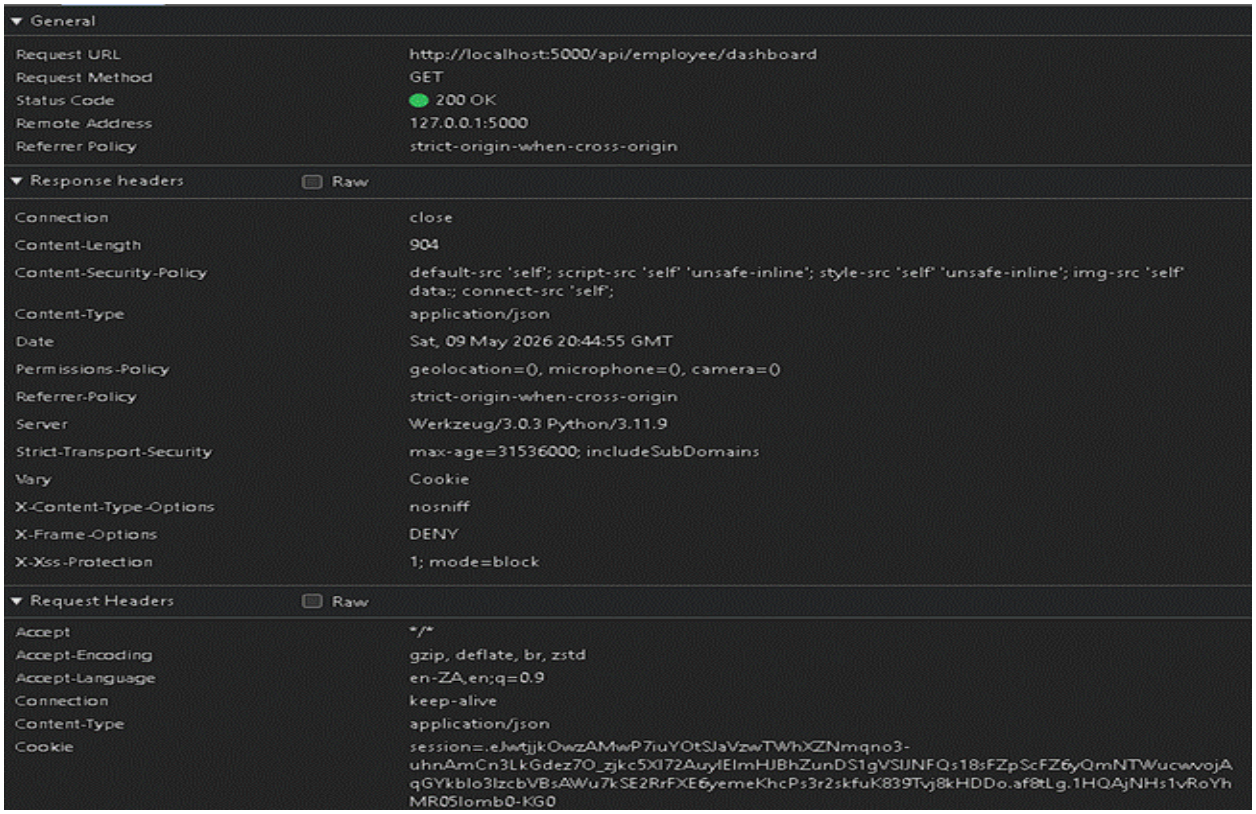


Figure 27

What makes this work in the code:

```
after_request hook in app.py  — applies all six security headers to every response: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Content-Security-Policy, Strict-Transport-Security, Referrer-Policy, Permissions-Policy
```

T08 — Error Handling and Information Disclosure (WSTG-ERRH-01)

We configured Burp Suite to listen on port 8080 as a proxy and routed our browser traffic through it. After logging in, we deliberately submitted incorrect credentials to trigger a failed login attempt, then examined the full HTTP response for that request in Burp Suite's HTTP history.

The login response returned a 401 status code with the body: {"message": "Invalid username or password", "success": false}. This message does not reveal whether the username exists, whether the password was incorrect, or whether the account is locked — the response is intentionally generic, giving an attacker no signal about

which part of the check failed. The Server response header returned was PaySecure rather than the framework name or runtime version, meaning the application does not expose any information about the underlying technology stack through its responses.

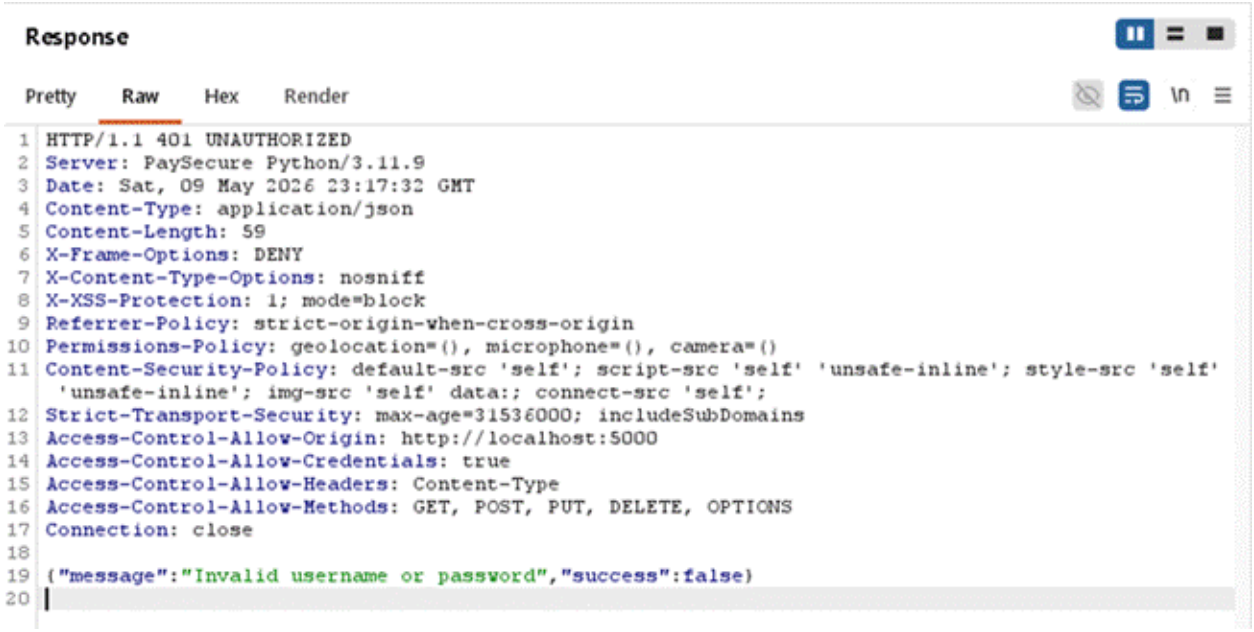


Figure 28

What makes this work in the code:

HideServerHeader class in app.py — subclasses WSGIRequestHandler and overrides server_version to return "PaySecure", replacing Werkzeug's default framework fingerprint at the HTTP transport layer

6.3.5.2 Naive Site

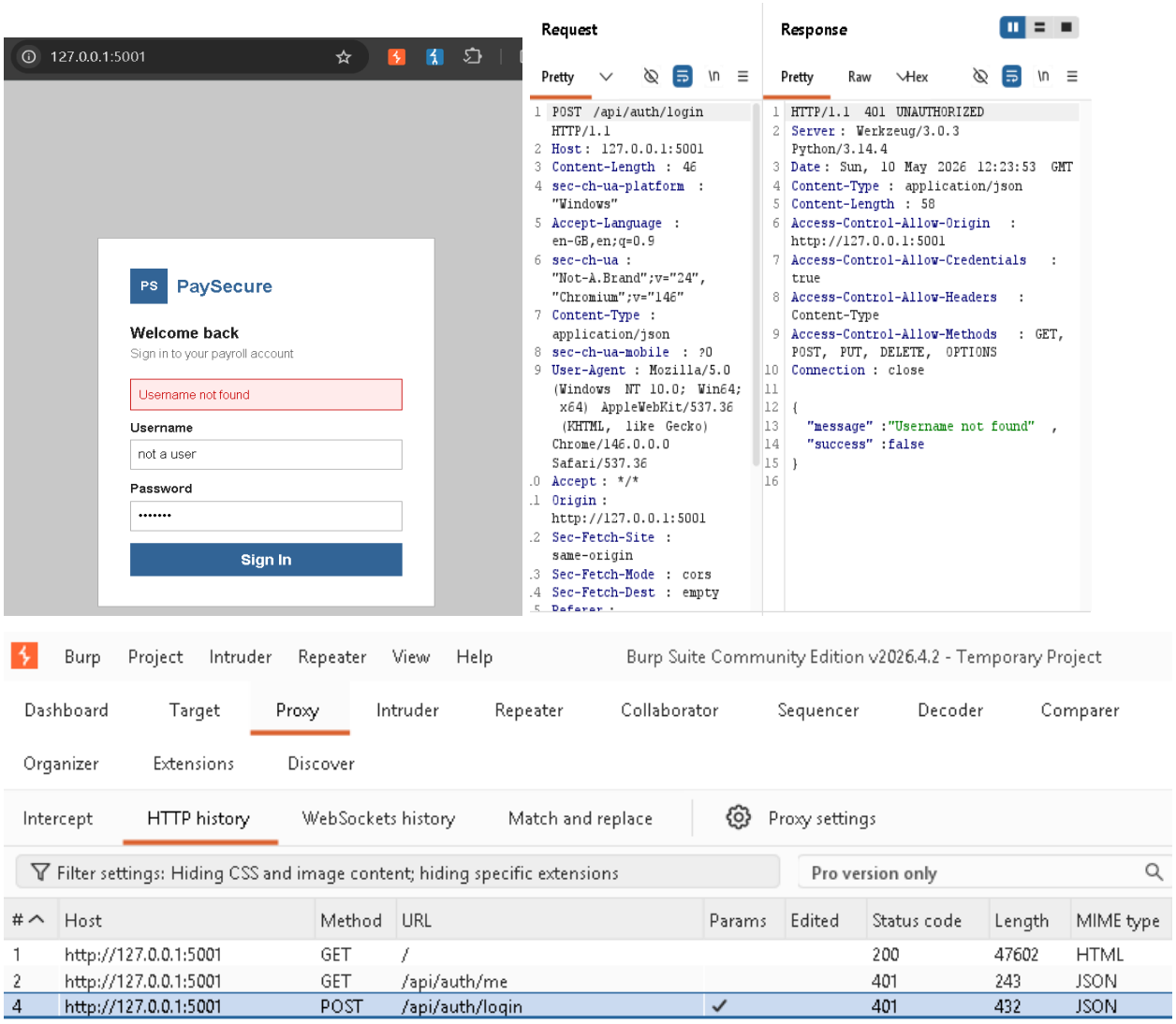
T07 — Security Headers and Clickjacking (WSTG-CONF-07) — Fail

The naive application failed T07. CVSS v3.1 score: 7.3 (High), vector AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L. DevTools inspection confirmed that all six headers applied by the secure site were absent: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, and Permissions-Policy.

The failure comes from the removal of the @app.after_request hook (section 5.6.2). Flask sends no security headers by default. Each missing header maps to a class of browser-side defence: X-Frame-Options blocks clickjacking, CSP constrains script execution and origins, nosniff blocks MIME confusion, and HSTS enforces HTTPS. The high severity score is because these gaps increase the impact of vulnerabilities elsewhere in the application. The absence of CSP, in particular, raises the impact of T04's stored XSS by removing the runtime restriction on what an injected script can contact.

T08 — Error Handling and Information Disclosure (WSTG-ERRH-01) — Fail

The naive application failed T08. CVSS v3.1 score: 5.3 (Medium), vector AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N. Burp Suite capture of a failed login showed two disclosures. The response body returned "Username not found" rather than a generic message, and the Server header advertised Werkzeug/3.0.3 Python/3.14.4.



The verbose error is the same disclosure as T06, observed at the transport layer. The framework fingerprint comes from removing the secure site's HideServerHeader class (section 5.5), which overrides server_version to return PaySecure. The two leaks tell an attacker which usernames are valid and which framework version the application is running. The version information is enough to query a vulnerability database for known issues against those specific versions.

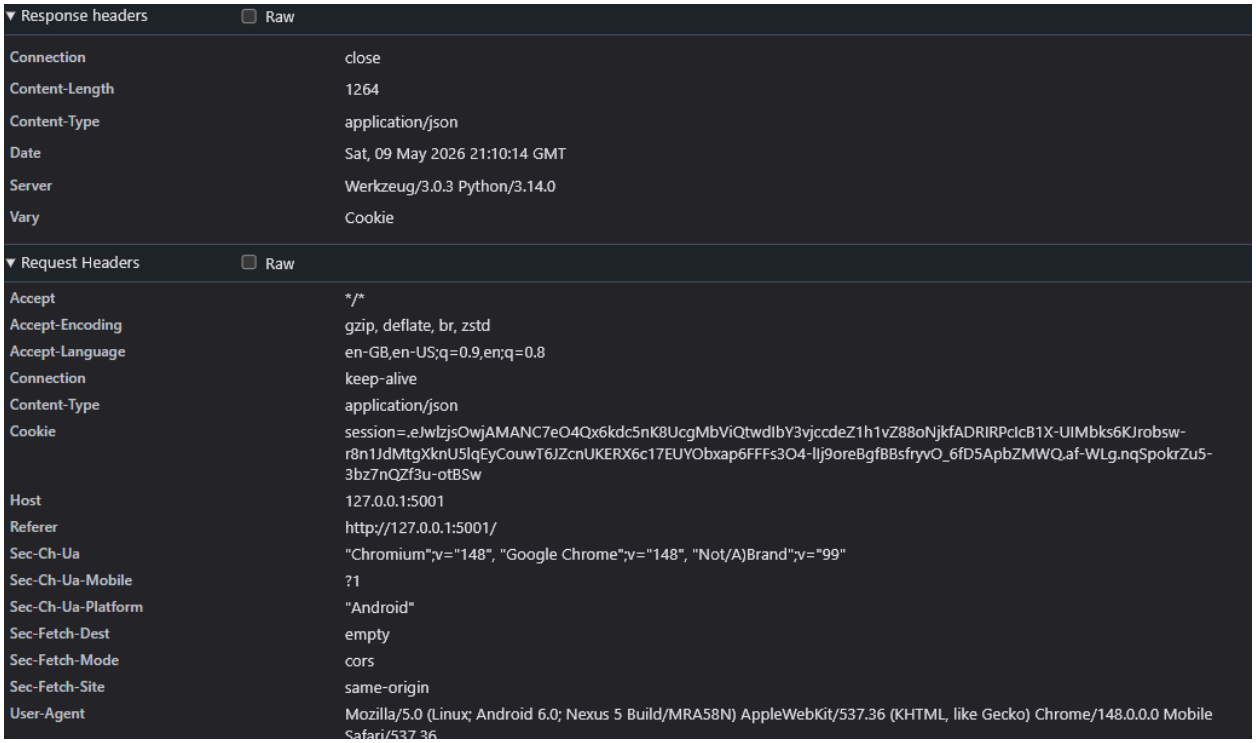
6.3.5.3 Vulnerable Site

T07 — Security Headers and Cookie Attributes (WSTG-CONF-07)

The response headers, located in the DevTools under the network heading section were directly inspected following the reloading of the site for inspection.

This check confirmed that all security headers were not enabled showing that the insecure site is far more susceptible for a multitude of attacks including

1. Xss attacks (Content-security-policy)
2. None HTTPS attacks (Strict-Transport-security)
3. Clickjacking attacks (X-Frame options)
4. Mime sniffing attacks (X-Content-types)
5. Browser built in XSS Protection



▼ Response headers ☐ Raw

Connection	close
Content-Length	1264
Content-Type	application/json
Date	Sat, 09 May 2026 21:10:14 GMT
Server	Werkzeug/3.0.3 Python/3.14.0
Vary	Cookie

▼ Request Headers ☐ Raw

Accept	*/*
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-GB,en-US;q=0.9,en;q=0.8
Connection	keep-alive
Content-Type	application/json
Cookie	session=.eJwlzjsOwjAMANC7eO4Qx6kdc5nK8UcgMbViQtwdIbY3vjccdeZ1h1vZ88oNjkfADRIRPcIcB1X-UIMbks6KJrobsw-r8n1JdMtgXknU5lqEyCouwT6JZcnUKERX6c17EUYObxap6FFFs3O4-llj9oreBgfBBsfryvO_6fD5ApbZMWQ.af-WLg.nqSpokrZu5-3bz7nQZf3u-otBSw
Host	127.0.0.1:5001
Referer	http://127.0.0.1:5001/
Sec-Ch-Ua	"Chromium";v="148", "Google Chrome";v="148", "Not/A)Brand";v="99"
Sec-Ch-Ua-Mobile	?1
Sec-Ch-Ua-Platform	"Android"
Sec-Fetch-Dest	empty
Sec-Fetch-Mode	cors
Sec-Fetch-Site	same-origin
User-Agent	Mozilla/5.0 (Linux; Android 6.0; Nexus 5 Build/MRA58N) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Mobile Safari/537.36

Figure 28

The absence of these headers shows that the insecure application provides no client-side protections, each missing security header increases the potential attack surface of the site.

T08 — Verbose Error Messages / Information Disclosure (WSTG-ERRH-01)

This tests the security of the application and testing how it handles error throws and what information is returned to the user.

This is important as the user can gain information on the system structure and behavior.

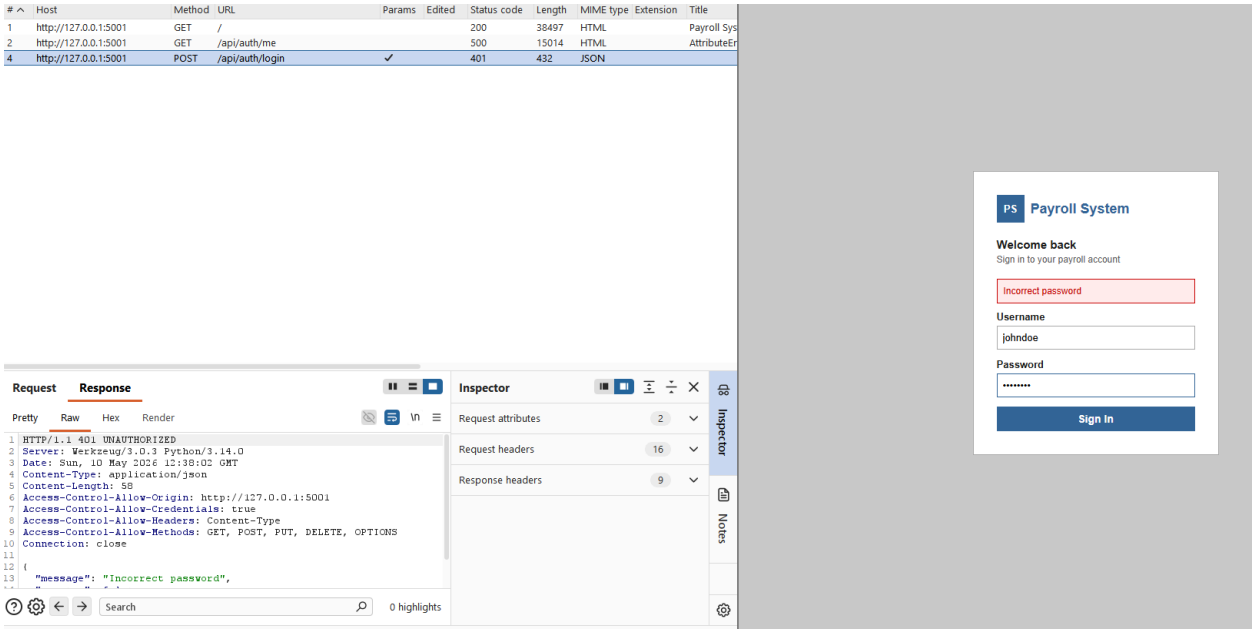


Figure 29

Burp suite was used, incorrect login information was entered so that the error handling could be observed.

The system threw a “401” status code meaning it was denied.

Upon inspection of the response headers you can see the sensitive information that should not normally be viewable to possibly malicious users.

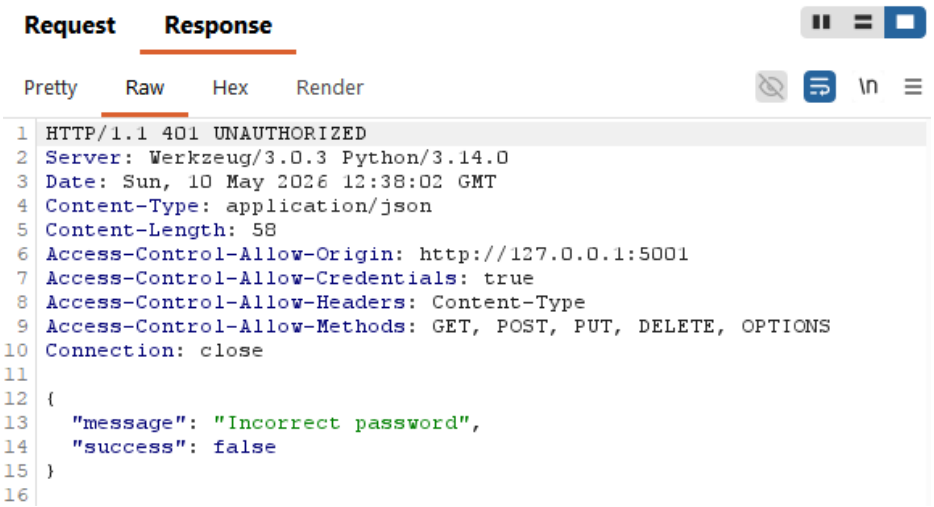


Figure 30

This result confirms that the insecure apps error-handling discloses actionable technical intelligence to any user that can inspect the HTTP traffic.
If a user is able to gain access to the exact server software name and language versions allows attacker to gain further information and reconnaissance, allowing the attacker to be much more accurate in their next targeted attack.

6.3.6 A07 — Identification and Authentication Failures

6.3.6.1 Secure Site

T09 — Two-Factor Authentication Bypass (WSTG-ATHN-04)

This test didn't actually involve exploiting a technical vulnerability but rather relied on someone having already exposed an unsecured version of the password, which in this case was written on a sticky note left exposed by an employee. As predicted when testing the secure version, after entering the password it asked for a 2FA 6 digit TOTP code that it would wait on indefinitely for. Without this the session was not established and no routes that required authentication were available.

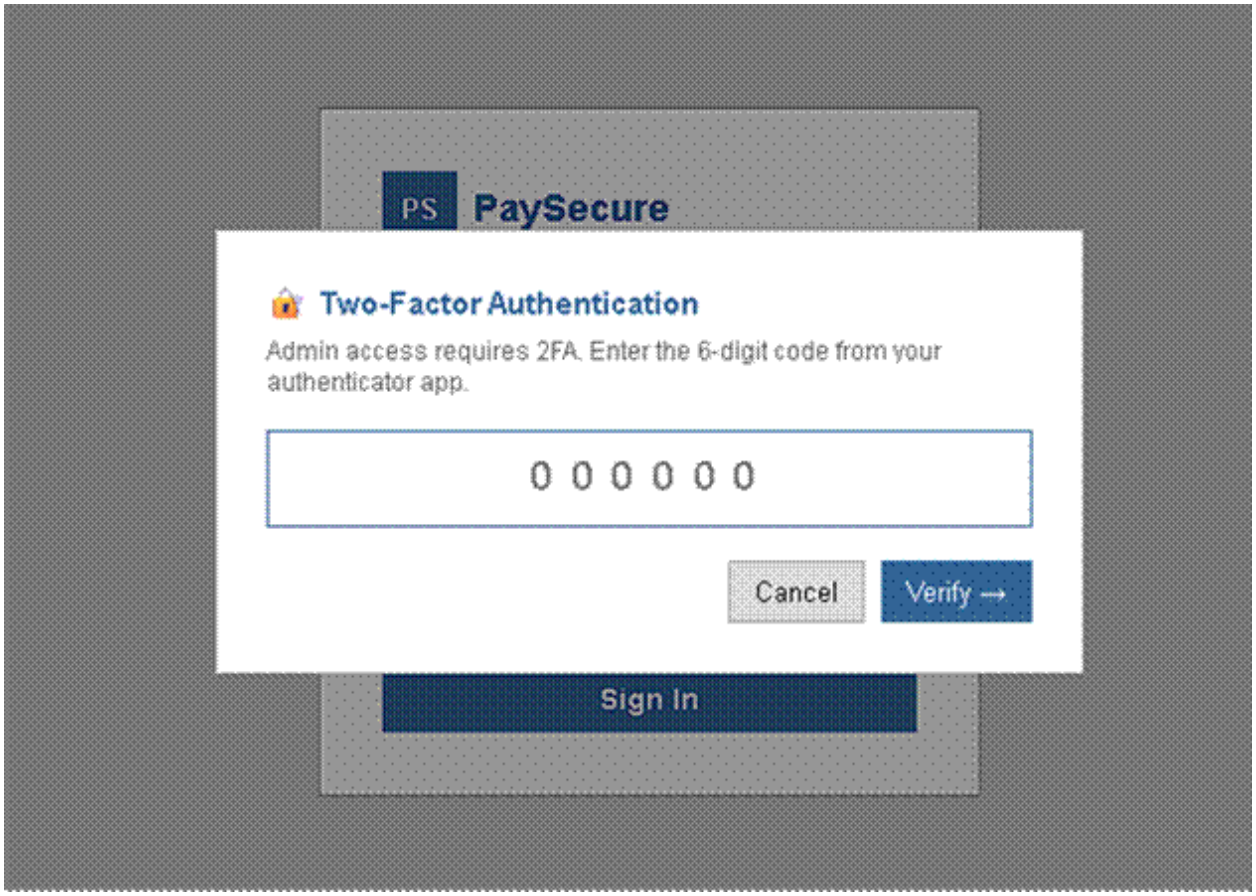


Figure 31

Additional testing was done on the 2FA gate. I tried inputting six incorrect codes on /api/auth/verify-2fa. The first five attempts returned a 401 Unauthorized error, and the sixth attempt returned a 429 Too Many Requests error.

Figure 32

The 2FA gate is strong on both fronts — the session is not created until the code is verified, and the verification step cannot be brute forced because the application locks out after five attempts.

What makes this work in the code:

```
pyotp.TOTP  — generates and verifies time-based one-time codes;
login_user() is only called after a valid code is confirmed

Rate limiting in routes/auth.py  — tracks failed 2FA attempts and
returns 429 after five consecutive failures
```

T10 — Account Lockout (WSTG-ATHN-03)

In a password cracking attack, a wordlist of commonly used passwords was submitted against the johndoe login page using Burp Suite Intruder, with each request sent in rapid succession. All requests were monitored for differences in status code and response length. Every attempt returned a 401 status code with a response length of 850 bytes, with no variation across the entire wordlist. There was no signal an attacker could use to distinguish one attempt from another.

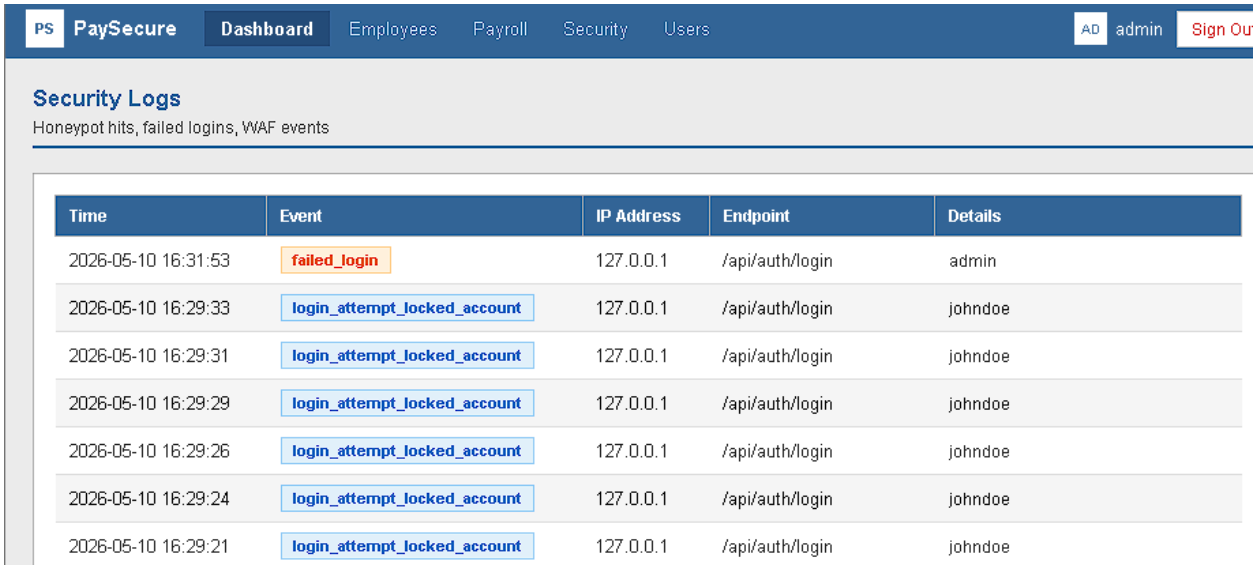


Figure 32

The security logs provided the fuller picture. After five consecutive failed attempts, the application locked the account and logged every subsequent request as login_attempt_locked_account rather than a standard failed login. The response to a locked account was designed to be indistinguishable from a wrong password — the same 401, the same generic message — so from the attacker's perspective there was nothing to indicate that a lockout had been triggered at all.

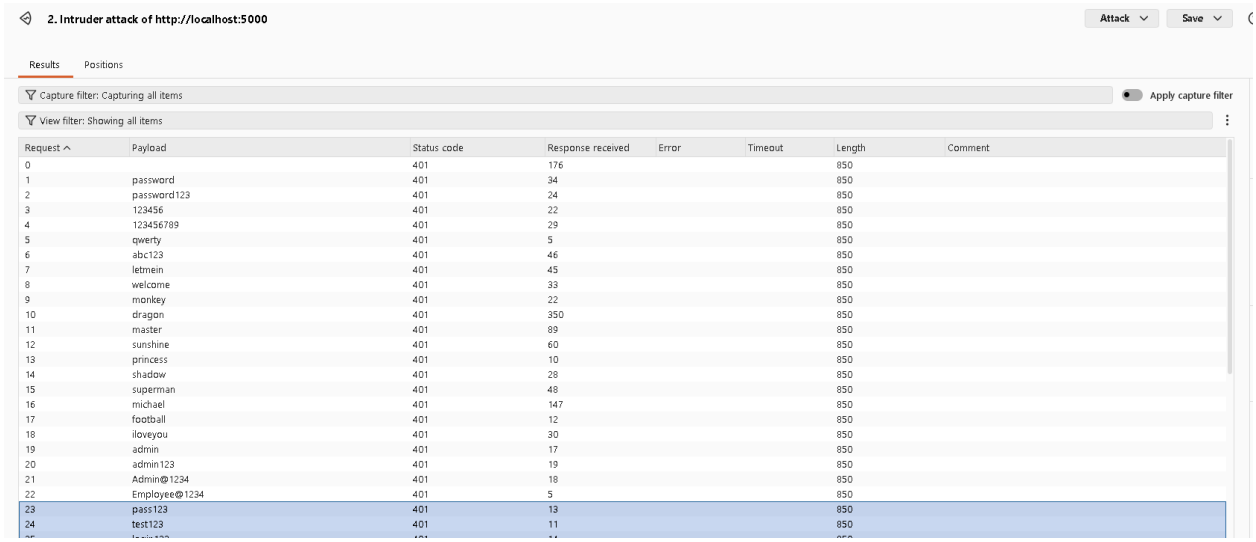


Figure 33

This result confirms the account lockout control is functioning as intended. The attacker was cut off after five attempts with no feedback and no way to determine from the responses alone whether the passwords were wrong or the account was locked, directly addressing the research question of whether authentication controls in the secure version meaningfully limit the impact of a credential attack.

What makes this work in the code:

```
routes/auth.py  — tracks failed_attempts per account; after 5 failures
sets locked_until to 15 minutes from the current time and returns 429
for all subsequent attempts until the lockout expires
```

6.3.6.2 Naive Site

T09 — Two-Factor Authentication Bypass (WSTG-ATHN-04) — Fail

The naive application failed T09. CVSS v3.1 score: 9.1 (Critical), vector AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N. Credentials assumed to have been obtained through a side channel, such as phishing or an exposed sticky note, were submitted to the login page. The application accepted the password and granted a session immediately, with no second-factor prompt issued.

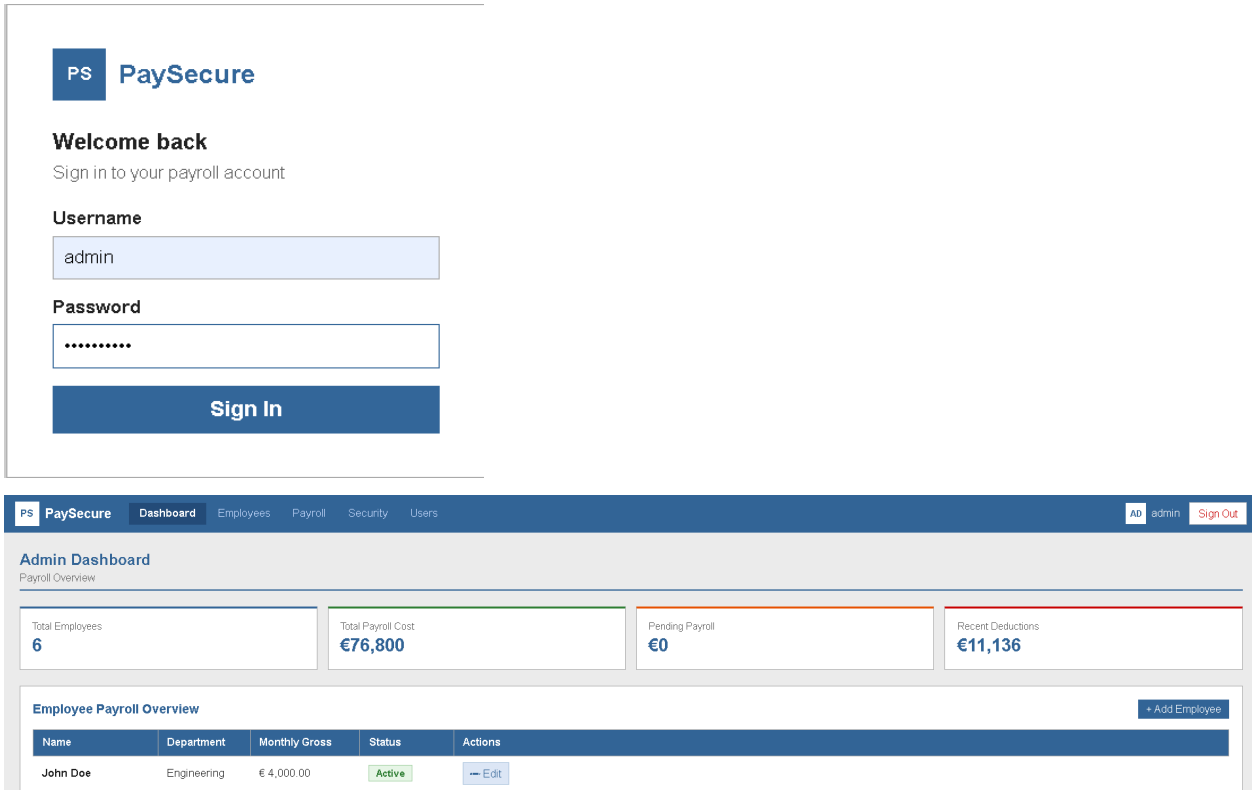


Figure 34

The failure is the absence of two-factor authentication rather than a bypass of one. The TOTP flow, the pyotp enrolment step, and the verification endpoint were all removed from the naive site (section 5.6.2) on the basis that a developer working without security awareness perceives 2FA as added friction. With no second factor in the login pipeline, a correct password is the only condition for session establishment, and any leaked credential is enough for full account takeover. The critical score reflects the position that single-factor authentication offers no defence in depth. Once the password is known by any means, the account is gone.

T10 — Account Lockout (WSTG-ATHN-03) — Pass

The naive application passed T10. A wordlist of seven common passwords loaded into Burp Suite Intruder produced seven 401 responses against the johndoe login. None of the passwords matched, and no successful login occurred.

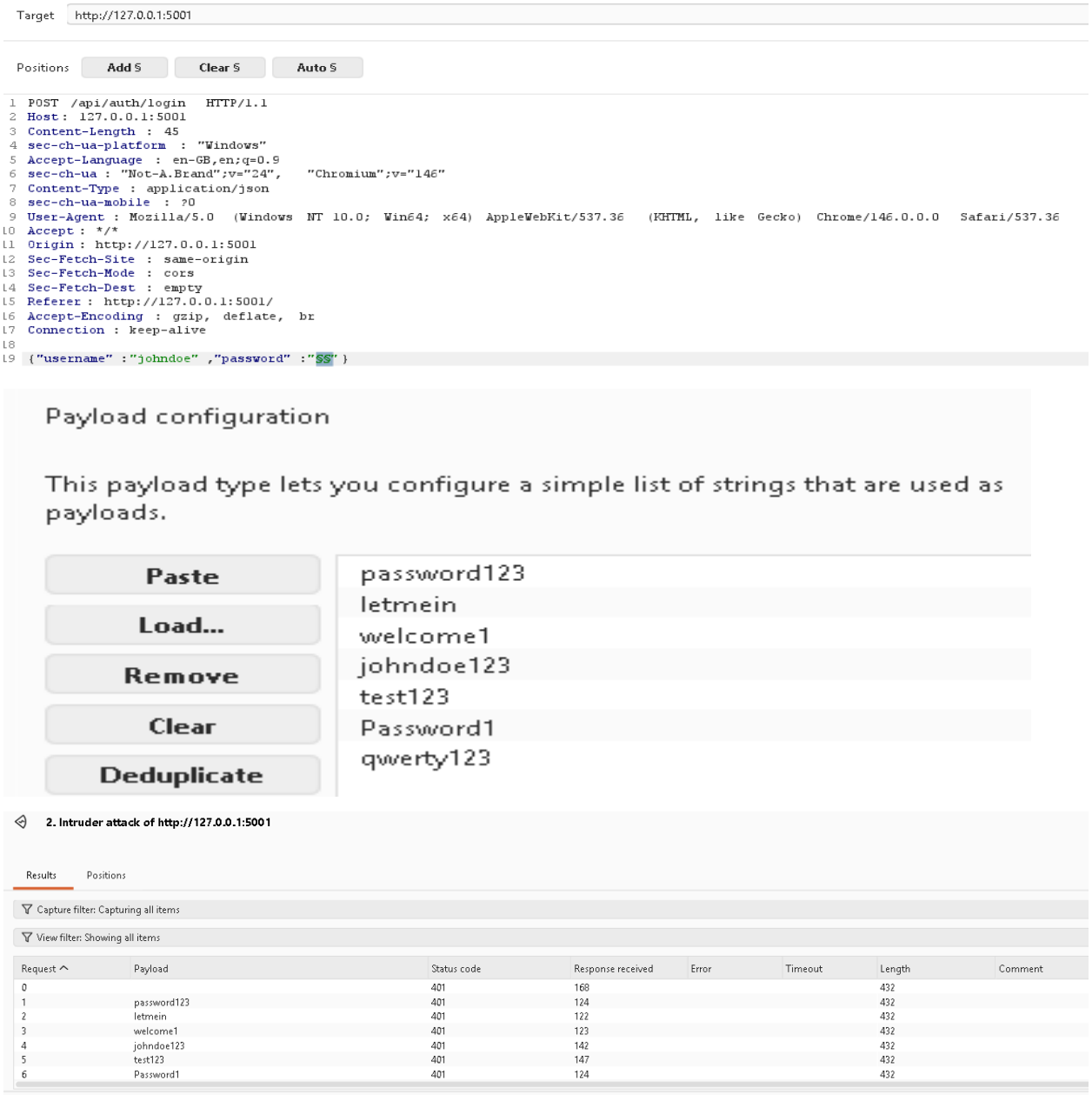


Figure 35

The naive site does not implement account lockout, the relevant logic was removed from the secure site (section 5.6.2), so the pass needs careful interpretation. What protected the account in this run was the wordlist itself rather than the application: the real password was not in the seven candidates tested. Had the wordlist contained the correct password, every attempt would have continued to be honoured with no rate limit and no lockout. The protocol classifies the test as Pass because no successful exploitation occurred, but the result is best read as

inconclusive on the underlying control. The contrast with T09 is therefore narrower than the headline counts suggest. Both tests target authentication weaknesses, both are critically vulnerable in principle, but only T09 produced a clear failure within its run.

6.3.6.3 Vulnerable Site

T09 — Two-Factor Authentication Bypass Testing (WSTG-ATHN-04)

The vulnerable site was tested as a control against the secure app, the assumption was made that a malicious user had gained access to an admin account.

This is to see if an account's login was compromised through phishing or other means , could the system still prevent the user accessing and maintaining access on a system.
2FA however did not stop the attacker from gaining access to the account , this is a failed result.

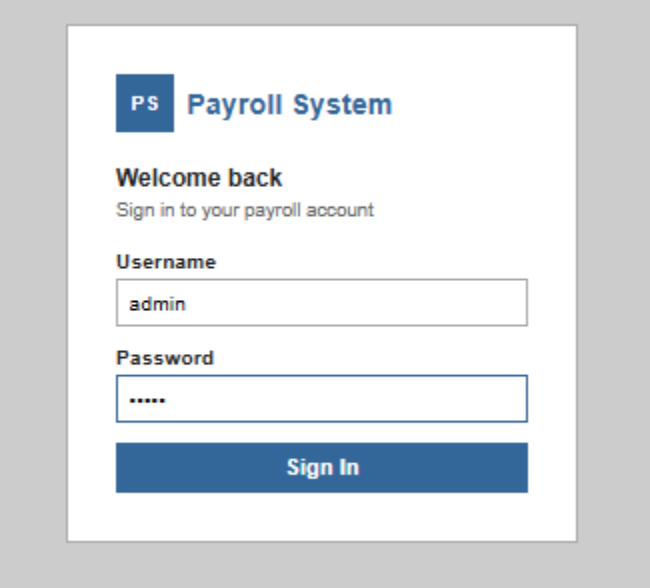


Figure 36

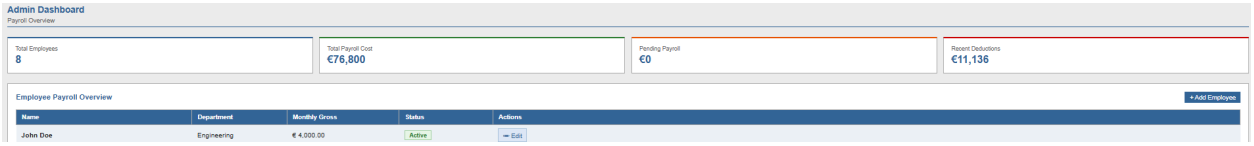


Figure 37

T10 — Testing for Weak Lock Out Mechanism (WSTG-ATHN-03)

Weak lock out Mechanism

This tests whether the app locks accounts after repeated failed login attempts.
Stopping automated brute force attacks.
This test was performed using burp suite.
Traffic was captured and the error was sent to intruder, from there a password list was built using frequently used words based around the user account and common inputs.

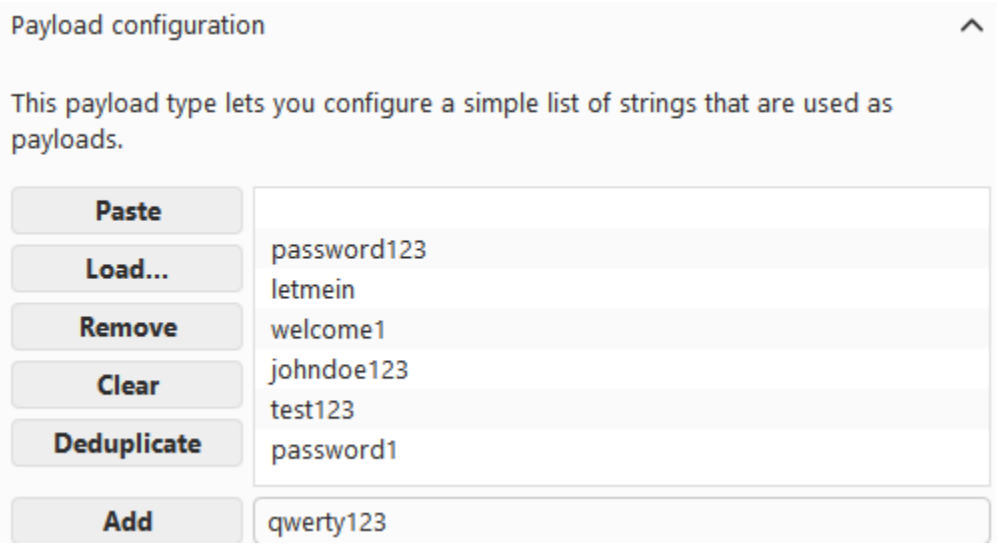


Figure 38

Request	Payload	Status code	Response receiv...	Error	Timeout	Length	Comment
0		401	4			432	
1		401	1			432	
2	password123	401	2			432	
3	letmein	401	2			432	
4	welcome1	401	2			432	
5	johndoe123	401	1			432	
6	test123	401	1			432	
7	password1	200	3			743	

Figure 39

A sniper attack was then launched using the password list , being unsuccessful 6 times before correctly guessing the password; this test fails as the session should deny the request if a user attempts too many times.

6.4 Cross-Cutting Themes

This section discusses the main patterns that appeared across multiple OWASP categories rather than focusing on each test individually. The results show that the difference between the three applications was not always simple. In some areas, the secure application was clearly stronger because it included deliberate security controls. However, in other areas, the naive application still had some protection because of the default behaviour of the framework and libraries used. The results also show that individual vulnerabilities can combine into larger attack paths, and that the most serious CVSS scores were not always linked to injection attacks.

Framework Defaults vs Explicit Security

One of the most important patterns in the results was the way framework defaults blurred the line between the secure and naive applications. The secure version included deliberate protections such as two-factor authentication, account lockout, security headers, generic error messages, role-based access control, and secure

session handling. These controls were added intentionally to reduce the application's attack surface.

However, the naive application was not completely unprotected. Some security protection came from the technologies used to build the system rather than from features added by the developer. SQLAlchemy handles database queries through parameterisation by default, reducing the risk of SQL injection without any explicit developer decision (SQLAlchemy, 2024). Similarly, Flask's default session cookie behaviour provided a degree of protection even when the developer had not specifically designed the system with security in mind. The naive version therefore represented a system where security had not been fully considered, but where some framework-level protections still remained — in contrast to the vulnerable application, where insecure behaviour was actively introduced through unsafe SQL handling, missing access control checks, weak cookie settings, and verbose error messages.

This finding is important when interpreting OWASP categories in a Flask context. A failed test does not always mean the framework is insecure, and a passed test does not always mean the developer made a strong security decision. As Alromaizan et al. (2026) note, modern frameworks increasingly embed security controls at the library level, meaning developers may inherit protections without awareness of them. Therefore, when analysing web security in modern frameworks, it is important to separate explicit security controls from protections that exist because of framework defaults.

Vulnerability Chains

Another clear theme was that vulnerabilities should not be viewed in isolation. Some weaknesses may appear limited on their own but become much more serious when combined with other flaws. This was especially visible in the access control tests.

T01 and T02 showed how one weak area can lead into another. In T01, the vulnerable application allowed a normal user to access administrator-level functionality. When combined with T02, the impact became wider — changing object identifiers in the URL exposed payslip records belonging to other users. Together, these flaws demonstrate how weak route-level access control and weak object-level access control can create a broader compromise of the system. Huang et

al. (2024) observe that access control vulnerabilities tend to appear in clusters precisely because authorisation logic applied only at the route level leaves every data resource below that route exposed, which aligns directly with what T01 and T02 produced.

The same pattern is visible in the relationship between XSS and insecure cookie settings. An XSS vulnerability is serious on its own, but it becomes more damaging if session cookies are also readable by JavaScript. Ahmad, Casarin and Calzavara (2023) found that client-side confidentiality of session tokens depends on the combination of HttpOnly and content policy controls, and the T03 and T04 results support this — where both protections were absent in the vulnerable application, the attack surface was significantly wider than either vulnerability alone would suggest. The secure application reduced this risk through layered controls: input filtering reduced the chance of script execution, while HttpOnly cookies reduced the impact if execution did occur, demonstrating why defence-in-depth remains the recommended approach to web application security (Mateo Tudela et al., 2020).

Severity Weighting

The CVSS scores also showed an important pattern. Although injection attacks are often seen as the most serious web application risk, the results suggest that access control weaknesses can be just as damaging, especially in a payroll system. SQL injection did receive the maximum score of 10.0 in T05, but access control failures in T01 and T02 scored 8.5 and 6.5 respectively, and the cryptographic failure in T03 scored 9.6 — all comparable to or exceeding several injection scores.

This matters because SQL injection is frequently treated in the literature as the primary example of a critical web vulnerability (OWASP, 2021). The results from this project show that broken access control can have an equally significant impact, and in some cases requires far less technical sophistication to exploit — in the vulnerable application, a normal user could access admin-level data by visiting an endpoint directly, with no payload required. For a payroll system specifically, this kind of failure is particularly serious: the data exposed includes employee names, salary information, payslip records, and personal details, carrying technical, financial, and legal consequences. The CVSS scores made this concrete by allowing the actual impact of each failure to be compared directly rather than assuming that

category determines severity, and they support the finding that access control should be weighted at least as heavily as injection when threat modelling for data-heavy applications.

Security Testing

Each of the 10 tests have been performed manually against all 3 variants of the application. All applications were developed and tested using a consistent set of tools and a shared testing protocol agreed across the team. Each test was manually executed with browser traffic routed through ZAP via FoxyProxy. Burp Suite Community Edition was used for T08 and T10, where raw response header inspection and Intruder-based payload delivery were required. Browser DevTools were also utilised as part of preliminary diagnostic efforts. The test cases were run in the exact same order from T01 to T10 and screenshots were captured throughout as evidence. The corresponding CVSS v3.1 scores were determined using the NIST calculator for every recorded failure, based on access and impact as observed during testing rather than theoretical worst-case scenarios.

Test Rationale and Coverage

The 10 selected tests were not picked arbitrarily. Each one corresponds to a specific vulnerability scenario from the OWASP Web Security Testing Guide v4.2, and were chosen to provide meaningful coverage of the categories most appropriate for payroll applications containing personal and financial information. All tests were chosen based on the criteria that each test was capable of producing a meaningfully different result across the three application versions. If a test would pass or fail identically regardless of security posture, it would add data points without adding insight.

The two tests mapped to A01 Broken Access Control, T01 and T02, were prioritised first since access control is the highest-ranked category in the OWASP Top 10:2021 and a payroll system is a high-value target for privilege escalation. An attacker who can reach admin functionality or access another employee's payslip does not need to exploit any other vulnerability, as the data is simply accessible. Two tests were included for this category rather than one to distinguish between vertical escalation, where a low-privileged user attempts to reach admin-level routes, and horizontal escalation, where a user tries to view a peer's record. These are distinct attack patterns that require distinct controls to prevent, and testing both provides a more complete picture of how well access control was implemented across the three versions.

T03 covering A02 Cryptographic Failures focused on session cookie configuration rather than other cryptographic scenarios such as password storage strength. Cookie theft is a runtime attack that can be tested directly against a live application, making it a more practical choice than attempting to evaluate password hashing strength, which would require database access and offline cracking outside the scope of this study. Session cookie configuration was also chosen because a misconfiguration here significantly increases the impact of a successful script injection, creating a direct link to T04.

The A03 Injection test was conducted using two test components, T04 and T05, which tested for Cross-Site Scripting and SQL Injection respectively. In the secure application these two types of injection were addressed in different ways, including a WAF and Content Security Policy for XSS, and ORM parameterisation for SQL injection. By testing both injection types separately, it was possible to evaluate each control independently rather than assuming that resistance to one form of injection implied resistance to the other.

T06 covering A04 Insecure Design focused on account enumeration because it is a category that is easy to overlook during development. Unlike injection or access control failures, insecure design vulnerabilities are not caused by a single line of poor quality code. They emerge from response logic that feels natural to implement. T06 was included in the test suite to cover a category that naive developers are prone to fail, as shown by Maceiras et al. (2024), who found that 93.7% of commercial services assessed were vulnerable to at least one form of account enumeration.

A05 Security Misconfiguration was covered by T07 and T08. T07 addressed the absence of HTTP security headers and clickjacking protection, which are configuration-level omissions rather than runtime exploits. T08 addressed information disclosure through response headers, chosen because there is passive risk in every response the application sends, which is observable through traffic inspection without any authentication required.

A07 Identification and Authentication Failures was covered by T09 and T10, designed to validate the implementation of two-factor authentication and account lockout respectively. They target the two main weaknesses in authentication systems, specifically inadequate second-factor requirements and unlimited login attempts, which together represent the minimum standard for authentication security in a system where credential compromise is a realistic threat. For a payroll application available to all employees within an organisation, the risk of a credential being stolen or guessed is not theoretical, and these tests measure how much damage that scenario causes under each security posture.

The remaining four OWASP Top 10:2021 categories were excluded from the test set as they fall outside the scope of this study:

- **A06 Vulnerable and Outdated Components** — This vulnerability requires a source code audit of the dependency rather than runtime testing.
- **A08 Software and Data Integrity Failures** — This does not apply to this Flask application.
- **A09 Security Logging and Monitoring Failures** — This test requires backend log access and long-term monitoring that a single session test cannot provide.
- **A10 Server-Side Request Forgery** — Requires URL-fetching capability which none of the three versions implement.

T01 — Vertical Privilege Escalation (WSTG-ATHZ-02)

The purpose of this test is to evaluate whether an individual with low-privilege access (an employee) can access parts of the application intended to be accessed by administrators only. Using an employee user account, attempts are made to directly access routes that are restricted to administrators such as employee management

and security logs. Additionally, attempts are made to inject roles via the browser console to test for potential client side privilege escalation by an attacker. The tester assessing for A01: Broken Access Control is evaluating if the application enforces access restrictions on the server level regardless of what is being presented to the user in the application's interface.

T02 — Unauthorised Function Level Access (WSTG-ATHZ-04)

This test also covers the A01 Broken Access Control vulnerability, and checks whether an authenticated employee can access other users' data by making requests outside of the normal UI flow. While logged in as a normal employee, the tester makes a fetch request from the browser console pointing to the employees endpoint for the admin users. The tester is checking to see if the server does role checking correctly, and whether it will return the correct data based on the request made, and the credentials that are set.

T03 — Cookie Security and Session Handling (WSTG-CRYP-03 / WSTG-CLNT-12)

This black box test aims to fulfill the requirements of A02 Cryptographic Failures by testing the session cookie that is generated for the user after a successful login. The test manually logs in and then uses the browser DevTools to check the attributes of the session cookie, primarily if HttpOnly and SameSite attributes have been correctly set to prevent theft by script and unauthorized transfer across sites. The web application passes this test if the cookie attributes have been set to lower risk sufficiently.

T04 — Cross-Site Scripting (WSTG-INPV-01)

The test aims to evaluate if an application follows the mitigation and protection guidelines for A03: Injection, by ensuring adequate sanitisation of user input both during processing and rendering. For this test, the following payload `<img src=x onerror=alert('XSS')>` was injected into the employee name field in the admin panel. This payload does not rely on a script tag making it likely to bypass basic filtering. The goal is to verify whether the payload has been neutralised prior to insertion into the database or simply reflected back to the browser.

T05 — SQL Injection (WSTG-INPV-05)

This test evaluates if the code under test properly handles and sanitizes user input for the parameter username for the action login. This test specifically targets A03: Injection — injection of SQL syntax via login form to manipulate database queries. Two payloads are needed. The first payload is ' OR '1'='1'# which can be used to test for authentication bypass. The second payload utilizes a UNION based query to test if database information can be extracted from the users table. The tester expects the application to treat the input as literal data not as SQL code.

T06 — Account Enumeration (WSTG-IDNT-04)

This test follows the brief outlined at A04: Insecure Design. The current test is designed to test whether a web application's login responses differ when asking if a submitted username exists. This can be checked by attempting to login with two incorrect passwords. The first password should be for a valid username, and the second incorrect password login attempt should be for a username that does not exist. The responses from each login attempt should be compared on status code, response body, and byte length, and there should be no differences to allow for guesswork of valid accounts.

T07 — Security Headers and Clickjacking (WSTG-CONF-07)

This test identifies whether a web application has not deployed security-related features that are available for most widely used application servers. This test will inspect HTTP responses for security-related headers, such as X-Frame-Options, Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, Referrer-Policy and Permissions-Policy. In addition, a clickjacking test page will be loaded to identify if the target application is able to be embedded in an iframe and what actions the application takes to inform the browser how to handle its content.

T08 — Error Handling and Information Disclosure (WSTG-ERRH-01)

For the purpose of this test, the web application and back-end database or software must be considered to be misconfigured if the existence of a web application framework, its version, and its runtime environment is apparent from examination of the HTTP responses to a failure. For example, if a test login attempt fails, Burp Suite can be used to capture the response from the server, particularly examining the Server response header and the body of the HTTP response. Typically, web application framework software will populate a Server response header with a

server name and version details. In this test, the focus is on whether any of the responses to the error test reveal information about the web application's underlying framework, database software, runtime environment or specific version numbers.

T09 — Two-Factor Authentication Bypass (WSTG-ATHN-04)

This test targets A07 Identification and Authentication Failures. The goal of this test is to determine whether or not a compromised password alone can be used to grant an attacker full access to the application. In instances with two-factor authentication, the tester will attempt to bypass the verification step using a known set of credentials. Furthermore, the tester will attempt to login with known credentials after submitting multiple incorrect TOTP codes in succession. The tester is looking for instances in which an application allows a session to be established prior to meeting all of the required authentication factors.

T10 — Account Lockout (WSTG-ATHN-03)

This test targets A07 Identification and Authentication Failures. It assesses whether the application under test will enforce a limit on the number of consecutive failures an attacker has when attempting to log in. The test uses Burp Suite Intruder to perform a brute force attack against a known account using a wordlist of common passwords. The test identifies whether the application will cut off login attempts after a certain number of failures. Additionally, the test identifies whether any of the responses from the application provides the attacker an indication of whether the password that was provided was valid or not.

7.0 Conclusion

This study set out to examine how meaningfully the security posture of a web application changes depending on the development decisions made during its construction. Three functionally identical payroll applications were built and tested against ten structured security tests drawn from the OWASP Web Security Testing Guide v4.2, each mapped to a corresponding category in the OWASP Top 10:2021. The findings were unambiguous. The secure application passed all ten tests. The vulnerable application failed all ten. The naive application, which represents the more realistic and instructive case, produced a mixed set of outcomes that reveals

how far a well-intentioned but under-informed implementation falls short of an acceptable security baseline.

The results answer the research questions with reasonable confidence. The secure application demonstrated that a small set of deliberate implementation choices, applied consistently across authentication, input handling, session management, response configuration, and access control, is sufficient to block every attack vector tested. No single payload reached its intended target. The naive application is the more instructive failure case because it is not entirely wrong — it implemented some controls in some areas, and those controls produced passing results where they were applied. The failure pattern is not one of a developer who did not care about security, but of one who did not apply it systematically, which is the more common real-world scenario. The vulnerable application failed every test as designed, establishing the floor against which the other two versions are measured.

Taken together, the data supports the conclusion that security controls need to be applied at every layer, and that gaps in any single layer produce failures that are not compensated for by controls applied elsewhere. The most consistent pattern across the results is that proactive controls — those which intercept a request before it reaches application logic, or limit what the response reveals — are more reliable than reactive ones that depend on the attacker behaving predictably.

7.1 Future Work

The scope of this study was bound to allow for depth within the constraints of the project. The most immediate limitation is that all testing was conducted in a local development environment over HTTP, meaning controls such as the Strict-Transport-Security header and the Secure cookie flag were present but inactive. A future study could deploy each version to a staging environment with a valid TLS certificate and repeat the full test suite under production conditions.

The test suite could also be extended to cover OWASP categories excluded here, particularly A06 Vulnerable and Outdated Components and A08 Software and Data Integrity Failures, both of which are relevant to a payroll application dependent on third-party libraries. Finally, a longitudinal study tracking a single application through successive stages of security hardening — rather than comparing three fixed endpoints — could provide more granular data on which individual controls

produce the largest reduction in attack surface and in what order they are most effective.